

Montar entorno de pruebas

Bienvenidos a esta nueva sesión en la que vamos a hablar de diferentes entornos.

Vamos a hablar de entornos que vamos a tener que instalar en nuestra máquina virtual a la hora de poder practicar y poder entender mejor las vulnerabilidades que nos vamos a encontrar en las aplicaciones web.

Y para ello vamos a instalar tres entornos que vamos a utilizar a lo largo de los vídeos, que son DVWA, Web for Pentester, tanto la versión 1 como la 2 y Juice Shop.

Al final estas máquinas son máquinas virtuales o programas que vamos a instalar dentro de nuestra máquina, en mi caso Kali o el sistema operativo que vayáis a utilizar, que ya tienen configuraciones que por defecto son inseguras, existen vulnerabilidades en ellas, están desarrolladas aposta para que así sea y eso nos va a permitir entender las vulnerabilidades, aprender a saber cómo detectarlas y cómo buscar esos fallos en la lógica de las aplicaciones.

Así que nos va a servir para aprender y para mejorar todas nuestras capacidades a nivel de ciberseguridad, de realizar este hacking web sobre las diferentes aplicaciones que nos podamos encontrar de ahora en adelante.

También lo que nos van a proporcionar, por eso vamos a utilizar varias, sobre todo al principio vamos a utilizar DVWA y Web for Pentester, nos van a permitir jugar con diferentes tipos de vulnerabilidades y además con diferentes niveles de complejidad.

Vamos a pasar desde la vulnerabilidad más fácil de explotar y el por qué se explota así a ir complicándolo un poco porque vayamos poniendo medidas de seguridad.

Por lo tanto, tener también diferentes niveles de complejidad nos va a ayudar a mejorar en la forma de detectar y de explotar estas vulnerabilidades.

¿Por qué vamos a utilizar estos entornos y no vamos a utilizar una web normal que podamos encontrar en Internet? Porque no es legal.

Entonces vamos a partir de la base de que, para ejecutar estas pruebas, necesitamos tener el OK por parte del cliente y sin autorización no podemos realizar este tipo de pruebas de manera abierta. Esto es un disclaimer muy importante que tenemos que tener en cuenta.

Tenemos que saber que solamente podemos realizar este tipo de pruebas de seguridad cuando tengamos la autorización por escrito y firmada del responsable de la aplicación. Por eso a esto se les llama entornos controlados.

Y a día de hoy también existen otras plataformas que también se pueden utilizar para este fin, como puede ser HackTheBox, TryHackMe, etcétera.

Y lo que vamos a utilizar nosotros es simplemente para montarnos un pequeño laboratorio en local, para que tampoco sea necesario tener Internet, que sea controlado, que esté en nuestra máquina, entender lo que he dicho, que si no estaríamos incurriendo en problemas legales, no tenemos permiso para lanzar este tipo de ataques a cualquier web que nos encontremos si no tenemos la autorización expresa.

Y al ser entornos controlados vamos a controlar que no sale de nuestra red, que no exponemos tampoco nuestra red al exterior y que podemos realizar este tipo de pruebas porque somos nosotros mismos los responsables, es nuestra máquina, es nuestra red y son aplicaciones y máquinas virtuales preparadas específicamente para este objetivo.

Vamos a probar DVWA, que va a contar con las vulnerabilidades más comunes de auditorías web, de aplicaciones web.

Luego vamos a tener Web for Pentester, tanto 1 como 2, que son de pentesterlab, son gratuitas y vamos a instalarlas y vamos a utilizar las dos versiones, porque tenemos diferentes tipos de vulnerabilidades y muchos ejemplos de cada uno de los tipos. No vamos a hacer todos a través de vídeos, pero sí que se va a proponer muchos que hagáis vosotros de manera práctica.

Y lo que nos permite es desde el ejemplo 1 al 10, lógicamente va a ir ampliándose la dificultad, va a ir variando el tipo o la forma de explotar cada vulnerabilidad y

compaginando tanto la versión 1 con la 2 vamos a tener una amplia gama de vulnerabilidades a probar.

Y la última es Juice Soft y lo interesante es que está simulando un entorno real, una tienda real y esta es más libertad a la hora de buscar las vulnerabilidades, no están guiada como DVWA o como Web for Pentester y tenemos que buscar vulnerabilidades como si estuviésemos en un entorno real que es una tienda.

Así que son los tres entornos que vamos a montar en local para tenerlos preparados porque los vamos a ir usando a lo largo del módulo.

Con esto hemos visto que una buena manera de aprender es ponernos manos a la obra, ponernos a practicar y para ello, a pesar de que hay diferentes opciones, nos vamos a decantar por montarnos un entorno local con aplicaciones web vulnerables sobre las que podamos lanzar ataques, hacer pruebas, romperlas, volverlas a levantar dentro de un entorno controlado.

Instalación de DVWA

En esta sesión vamos a hablar del tema de DVWA, que es uno de los entornos controlados que vamos a instalar en nuestra máquina virtual para poder comenzar a hacer las pruebas sobre auditorías de hacking web, pudiendo descubrir nuevas vulnerabilidades, viendo cómo descubrirlas y cómo explotarlas.

Así que para ello las siglas de DVWA significan Damn Vulnerable Web Application es una aplicación creada de manera voluntaria para ser vulnerable. Se ha creado aposta con este fin para aprender de vulnerabilidades y de seguridad en aplicaciones web.

Tiene un GitHub donde tenemos todo su código fuente y donde podemos descargarlo para instalarlo en nuestro equipo.

Y se puede instalar de varias maneras una máquina virtual utilizando XAMPP y montando ahí el servidor web.

Se puede montar directamente también sobre Apache, dentro de nuestra Kali o Parrot, lo que estemos usando, o también con Docker.

En este caso yo lo voy a montar con Docker porque me parece que es la manera más rápida y más sencilla de que todo funcione y no haya ningún problema a la hora de instalarlo.

Instrucciones de instalación: Docker

Lo primero que tenemos que saber es si tenemos Docker instalado en nuestra máquina, en caso de no tenerlo instalarlo con estos comandos que tenemos aquí y probar que funciona:

```
sudo apt update
```

```
sudo apt install -y docker.io
```

```
sudo systemctl enable docker --now
```



```
sudo docker run hello-world
```

Debajo tenemos la respuesta de este run Hello World con Docker:

```
└─$ sudo docker run hello-world
[sudo] password for securiters:

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Y si nos responde esto quiere decir que nuestro Docker está correctamente instalado y funcionando. Así que vamos a comprobarlo.

Me voy a la máquina virtual de Kali, inicio sesión. Y aquí lo que vamos a tener que hacer, lo primero que se me conecte a la red, porque si vamos a tener que descargar cosas.

Yo aquí poniendo Docker no lo tengo instalado, así que tengo que instalarlo.

Haré un `sudo apt update` que no debería tardar mucho porque está bastante actualizada la máquina. Es lo primero que tenemos que hacer siempre cuando vayamos a instalar algo, hacer un apt update.

Lo siguiente sería un `sudo apt install -y docker.io`.

Una vez instalado podemos decirle que nos levante docker siempre cada vez que iniciemos el sistema. Para ello con `sudo systemctl enable docker --now` le vamos a decir que habilite docker y que además ya nos lo haga desde ahora mismo docker con un now y así ya nos lo levantará.

A partir de este momento yo ya tengo la herramienta de docker instalada y ahora lanzaremos ese hello world con docker para asegurarnos de que funciona.

Así que con `sudo docker run hello-world`, que al final docker es un contenedor, por eso no necesitamos hacer ninguna configuración a bajo nivel porque ese contenedor va a tener todo lo que necesita la aplicación para funcionar.

En este caso nos dice que no encuentra hello world en local, lo descarga y nos devuelve este mensaje de docker. Aquí nos lo ha descargado primero ha hecho un pull, tenemos aquí el pull completo y después ya nos lo ejecuta con este resultado. Eso quiere decir que tenemos nuestro docker preparado para funcionar.

Instrucciones de instalación: DVWA

```
sudo docker run -p 80:80 --name dvwa -t vulnerables/web-dvwa
```

```
sudo docker start dvwa
```

```
sudo docker stop dvwa
```

Lo siguiente ahora será instalar directamente nuestra aplicación de DVWA, para lo que tendremos que bajarnos el contenedor de docker de DVWA, ejecutarla en el puerto 80 de nuestro localhost y luego yo le he puesto un tag name para llamarlo y poderlo levantar y tirar.

Vamos a ver si funciona correctamente y vamos a descargarlo.

```
sudo docker run -p 80:80 --name dvwa -t vulnerables/web-dvwa
```

Aquí tendríamos que repetir `sudo docker run`, aquí le voy a decir que funciona en el puerto 80 con `-p 80:80`, el `--name` que yo he dicho que le voy a poner, vamos a ver si funciona `dvwa` y de dónde lo tiene que descargar con `-t`, pues este contenedor está en `vulnerables/web-dvwa`.

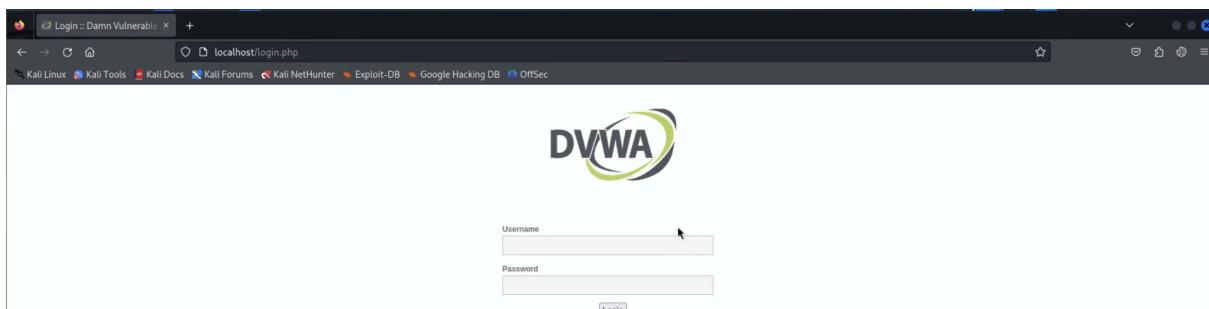
Le damos a enter y primero va a buscar la imagen, como no la tenemos en local, la va a descargar, la busca automáticamente de manera transparente para nosotros,

con ese nombre como le hemos indicado donde está el contenedor, es capaz de descargarlo.

```
Status: Downloaded newer image for vulnerables/web-dvwa:latest
[+] Starting mysql...
[ ok ] Starting MariaDB database server: mysqld.
[+] Starting apache
[....] Starting Apache httpd web server: apache2AH00558: apache2: Could not reliably determine the server's fully qualified domain name
, using 172.17.0.2. Set the 'ServerName' directive globally to suppress this message
. ok
=> /var/log/apache2/access.log <=
=> /var/log/apache2/error.log <=
[Thu Mar 14 10:13:48.777887 2024] [mpm_prefork:notice] [pid 306] AH00163: Apache/2.4.25 (Debian) configured -- resuming normal operation
[Thu Mar 14 10:13:48.777944 2024] [core:notice] [pid 306] AH00094: Command line: '/usr/sbin/apache2'
=> /var/log/apache2/other_vhosts_access.log <=
```

Cuando la descarga empieza a ejecutarla y fijaros, nos pone que está ejecutando una base de datos como MySQL, MariaDB, ha levantado también el Apache, el servidor web y ya tenemos el log escuchando.

Si esto ha funcionado correctamente y nos vamos al navegador, en nuestro localhost debería de estar funcionando, escribimos localhost en el navegador



Y aquí lo tenemos, la prueba de que está funcionando DVWA.

Voy a cerrar este docker, pero no nos deja, se ha quedado pillado. Hago un `netstat -putan` para ver qué puertos tengo abiertos.

```

Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:37629         0.0.0.0:*               LISTEN      -
tcp        0      0 192.168.111.139:46006   35.244.181.201:443      ESTABLISHED 11067/firefox-esr
tcp        0      0 192.168.111.139:56202   34.98.75.36:443         ESTABLISHED 11067/firefox-esr
tcp        0      0 192.168.111.139:34966   34.160.144.191:443      ESTABLISHED 11067/firefox-esr
tcp        0      0 192.168.111.139:36548   34.120.115.102:443      ESTABLISHED 11067/firefox-esr
tcp        0      0 192.168.111.139:48076   142.250.200.67:80       ESTABLISHED 11067/firefox-esr
tcp        0      0 192.168.111.139:38694   34.160.144.191:443      ESTABLISHED 11067/firefox-esr
tcp        0      0 192.168.111.139:40362   142.250.200.131:80      ESTABLISHED 11067/firefox-esr
tcp        0      0 192.168.111.139:59094   35.201.103.21:443       ESTABLISHED 11067/firefox-esr
tcp        0      0 192.168.111.139:42064   34.117.237.239:443      ESTABLISHED 11067/firefox-esr
tcp        0      0 127.0.0.1:80            127.0.0.1:33960         TIME_WAIT   -
tcp        0      0 192.168.111.139:59434   34.149.100.209:443      ESTABLISHED 11067/firefox-esr
tcp        0      0 192.168.111.139:38436   34.107.243.93:443       ESTABLISHED 11067/firefox-esr
tcp        0      0 192.168.111.139:51962   34.117.188.166:443      ESTABLISHED 11067/firefox-esr
tcp        0      0 192.168.111.139:50198   18.154.41.33:443        ESTABLISHED 11067/firefox-esr
tcp        0      0 192.168.111.139:50490   2.20.71.212:80          ESTABLISHED 11067/firefox-esr
tcp        0      0 192.168.111.139:60426   52.13.152.141:443       TIME_WAIT   -
tcp        0      0 192.168.111.139:40100   192.229.221.95:80       ESTABLISHED 11067/firefox-esr
tcp        0      0 192.168.111.139:46390   142.250.200.106:443     TIME_WAIT   -
tcp        0      0 192.168.111.139:32804   34.160.90.233:443       ESTABLISHED 11067/firefox-esr
tcp        0      0 192.168.111.139:38444   34.107.243.93:443       ESTABLISHED 11067/firefox-esr
tcp        0      0 192.168.111.139:59392   2.20.71.80:80           ESTABLISHED 11067/firefox-esr
tcp6       0      0 :::80                   :::*                     LISTEN      -
udp        0      0 192.168.111.139:68      192.168.111.254:67      ESTABLISHED -
udp        0      0 0.0.0.0:32908           0.0.0.0:*               11067/firefox-esr

```

Tenemos aquí el puerto 80, ¿DVWA sigue funcionando? Parece que sí, a lo mejor se ha quedado pillado ahí el docker.

Éste de aquí se ha quedado con TIME WAIT.

Vamos a probar con `sudo docker stop dvwa`, a ver si lo para, se puede tardar un poquito. Me parece que lo está ejecutando bien, ahora si me voy al localhost ya no estaría levantado.

Lo bueno de ponerle ese nombre, ése tag name con `--name`, es que si ahora le digo que lo levante poniéndolo `sudo docker start dvwa`, lo levanta y aquí no me ocupa la consola con el log que teníamos previamente.

Se supone que ahora ya estará levantado. Si vuelvo al navegador y le doy localhost estaría levantado. Importante, al ser puerto 80 si le ponemos https no funcionará, tiene que ser http.

Y así desplegamos la aplicación, con `sudo docker start dvwa` y paramos la aplicación con `sudo docker stop dvwa`.

Cositas interesantes:

Lo primero para acceder a esta aplicación, usuario es admin, contraseña es password.

La primera vez que iniciemos sesión nos va a aparecer Database Setup.

Mirándolo pues tenemos un montón de cosas.

Aquí hay una cosita en rojo que con el docker es lo que pasa, habría que modificar.

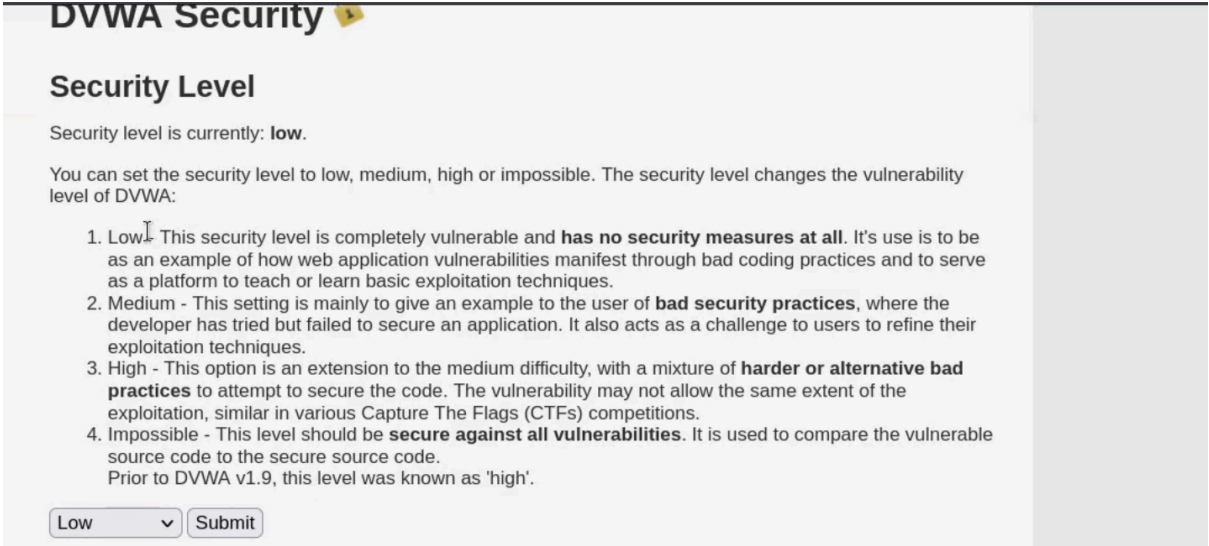
Vamos a necesitar crear una base de datos porque no la tenemos, así que le damos a “Create/Reset Database” y lo hace automáticamente.

Nos vuelve a echar fuera, volvemos a acceder, admin/password y ahora nos da la bienvenida a la aplicación web.

Y esta sería la aplicación web, Estamos logados como administrador.

Aquí abajo a la izquierda nos aparece la información de cuál es nuestro nombre de usuario, qué nivel de seguridad hemos puesto, el “low” es el más vulnerable y aquí tenemos a la izquierda el listado de las vulnerabilidades disponibles en esta aplicación Tendríamos para probar fuerza bruta, para probar SQL Injection, para probar Cross Site Scripting etc, diferentes funcionalidades, diferentes vulnerabilidades que nos van a permitir probar y mejorar las técnicas que vayamos conociendo.

DVWA security nos permite poner el nivel de seguridad que queramos y vamos a tener cuatro niveles.



The screenshot shows the 'DVWA Security' page. At the top, it says 'Security Level'. Below that, it states 'Security level is currently: low.' and explains that the security level can be set to low, medium, high, or impossible. A list of four levels is provided: 1. Low (completely vulnerable), 2. Medium (bad security practices), 3. High (harder or alternative bad practices), and 4. Impossible (secure against all vulnerabilities). At the bottom, there is a dropdown menu set to 'Low' and a 'Submit' button.

DVWA Security

Security Level

Security level is currently: **low**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. **Low** - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. **Medium** - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. **High** - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. **Impossible** - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.

Low Submit

El low todo es vulnerable, no hay ninguna medida de seguridad, el medium se han puesto medidas de seguridad pero no son las correctas, son malas prácticas a nivel de seguridad. El high, mejores medidas de seguridad, pero aún así nos las vamos a

poder saltar. Y el Impossible, que debería de ser seguro contra todas las vulnerabilidades, pero luego como todo aquí en principio todo es vulnerable.

Antes de esta versión el Impossible era el que se conoce como Hide, por si lo utilizáis de alguna otra versión previa, estaríamos en la 1.10 Development.

Con esto yo creo que más o menos tenemos nuestro DVWA funcionando, sería lo mínimo indispensable que tenemos que tener en este momento.

Tenemos ya un primer entorno controlado, que es DVWA montado local en nuestra máquina a través de Docker, es una buena manera para comenzar a aprender sobre vulnerabilidades en aplicaciones web, entenderlas e ir aprendiendo también técnicas de explotación. El objetivo es tener una aplicación web target para practicar de manera legal, porque para poder auditar y defender una aplicación web hay que entender cómo atacarla.

DVWA es un entorno controlado, tiene un gran número de vulnerabilidades que nos van a permitir aprender y mejorar nuestro conocimiento y diferentes niveles de complejidad.

Instalación Web for Pentester

Vamos a dejar configurado nuestro segundo entorno de aplicaciones web vulnerables y ese es el de Web for Pentester.

Así que lo que vamos a hacer va a ser conocer un poquito más estas vulnerabilidades.

Vamos a ver tanto la versión 1 como la versión 2, son dos máquinas virtuales que podemos descargar de manera gratuita de Pentester Lab, también están en VulnHub.

Tenemos aquí el recurso para descargarlo tanto de Pentester 1 como de Pentester 2:

https://pentesterlab.com/exercises/web_for_pentester/attachments

https://pentesterlab.com/exercises/web_for_pentester II/attachments

Me voy a ir al navegador, fijaros, la buscamos como pentesterlab, web for pentester versión 1. Además es la parte gratuita, tenemos hasta un curso, tenemos vídeos, tenemos archivos.

Y en los archivos lo que vamos a hacer va a ser descargarnos esta máquina virtual. Como veis no pesa prácticamente nada y lo mismo haremos con la versión número 2.

Iremos a files y nos descargaremos esta máquina virtual.

Yo ya he hecho los deberes, lo tenemos aquí descargado y lo siguiente que tendremos que hacer es importarlo dentro de nuestras máquinas virtuales, en mi caso con VMware.

Así que tenemos que decirle que queremos una nueva máquina virtual, decirle dónde tenemos esas ISO, y vamos a instalar primero la número 1, sin más, next, next, vamos a ponerle 8 gb.

Ahora sí, le damos a siguiente que finalice y perfecto, no necesitan prácticamente nada para funcionar, vamos a dejarlo que vaya funcionando, vemos que va funcionando y automáticamente estamos dentro.

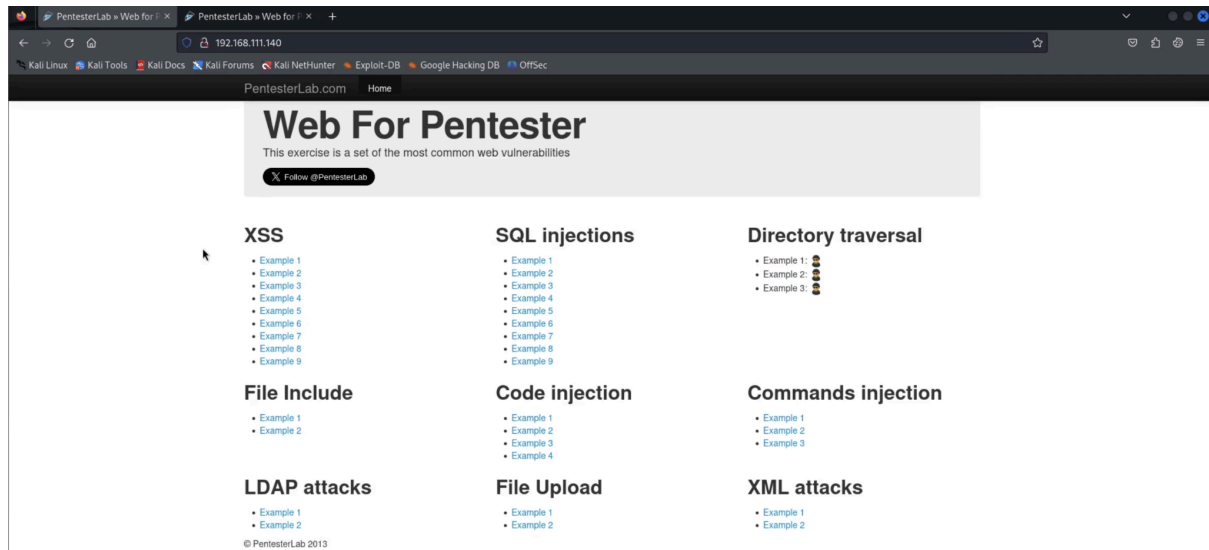
Aquí podríamos decir whoami y un ifconfig y tenemos la IP de la máquina, 192.168.111.140.

Esto mismo lo vamos a repetir, lo voy a hacer ya directamente con la segunda versión, lo mismo, browser y buscamos la versión 2, next, aquí voy a poner webfortester 2, que la otra no lo he cambiado, y lo mismo, 8 GB suficientes para que no me ocupe todo el equipo, está instalando.

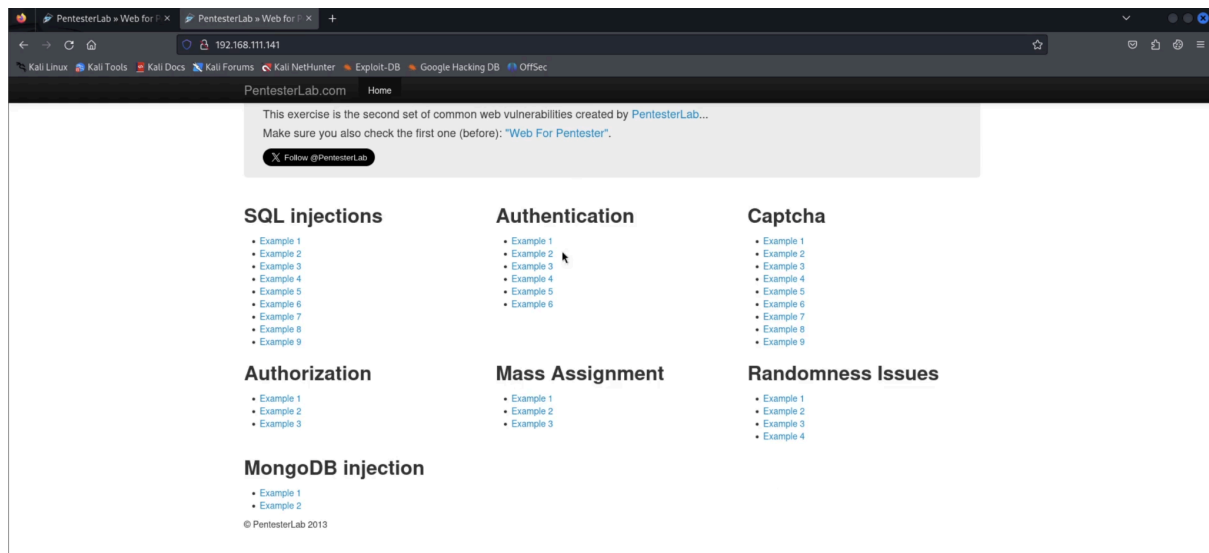
Y lo mismo, tendríamos también otra IP que será la 192.168.111.141.

Pues nos iríamos a nuestra máquina atacante, vamos a confirmar cuál es nuestra IP y tenemos la 192.168.111.139, así que será ir siguiendo a la 140 y a la 141.

Son webs que están en el puerto 80, así que vamos a acceder a la 140 web for pentester y a la 141 web for pentester 2.



Y fijaros, tenemos diferentes tipos de vulnerabilidades, por ejemplo para empezar con Cross Site Scripting, SQL Injection, Code Injection, File Injection, LDAP, File Upload, Command Injection.



Y sin embargo en la versión 2 tenemos cositas más de autenticación de autorización, cosas de captcha, de Mass Assignment, temas random y de MongoDB.

No vamos a ver todas, pero nos va a servir entre estas dos versiones de máquinas virtuales para hacer algunas de las pruebas que vamos a ir descubriendo a lo largo de las siguientes sesiones.

Al final para ver las vulnerabilidades web, ver cómo explotarlas y ver diferentes formas de explotarlo, que es nuestro objetivo ahora mismo, porque vamos a tener diferentes recursos para hacerlo.

A nivel de conclusiones, fácil, una manera buena de aprender.

Ya tenemos otro entorno preparado con vulnerabilidades para poder practicar todo lo que queramos de manera controlada, con las vulnerabilidades más comunes cubiertas tanto con la versión 1 como con la versión 2 y diferentes niveles de complejo.

Partiremos del ejemplo 1 y según vayamos incrementando el número, la dificultad irá incrementándose también.

Instalación de Juice Shop

En esta sesión vamos a tratar el tema de Juice Shop, que es otro de los entornos de prueba que necesitamos configurar de cara a poder hacer todas las prácticas que vendrán de ahora en adelante.

Este entorno es un entorno creado por OWASP, es otro de los proyectos que tiene en el que ha creado una web con diferentes vulnerabilidades pero con un punto de vista diferente a lo que por ejemplo Tiene DVWA o Web for Pentester.

Es una web que simula ser una tienda de zumos, una tienda de zumos real, es decir, no nos va a explicar o no nos va a indicar dónde hay cada vulnerabilidad, aquí hay un SQL injection, aquí hay un cross site scripting, no, sería simular una web que nos podemos encontrar dentro de una auditoría normal y el objetivo es que vayamos descubriendo vulnerabilidades.

Lo que sí que tiene, que es una cosa bastante interesante, es que está gamificada, tiene un dashboard secreto que una vez que lo conozcamos sí que nos va a ir diciendo cuántas vulnerabilidades hemos encontrado, de qué dificultad en base a un score interno que tiene, en base a estrellas, como de fácil o de difícil es explotar o encontrar esas vulnerabilidades.

A la hora de la instalación se puede hacer igual con código fuente o se puede levantar directamente con Docker, lo recomiendo porque es más rápido y va a funcionar a la primera.

Y los pasos a seguir, pues bueno, ya sabemos que tenemos Docker instalado, tendremos que instalar este Juice Shop, podemos hacer el pull, podemos hacer el run directamente, meterle el --name, y luego pues levantarlo, pararlo, etc. Aquí tenemos los comandos:

```
sudo docker pull bkimminich/juice-shop
```

```
sudo docker run --rm -p 3000:3000 --name juice  
bkimminich/juice-shop
```

```
sudo docker start juice
```

```
sudo docker stop juice
```

Y es una aplicación con vulnerabilidades que se va a levantar en el puerto 3000, por lo tanto para acceder tenemos que ir a ese puerto y asegurarnos de ir por HTTP, no por HTTPS porque sino no vamos a encontrar la página.

Browse to <http://localhost:3000>

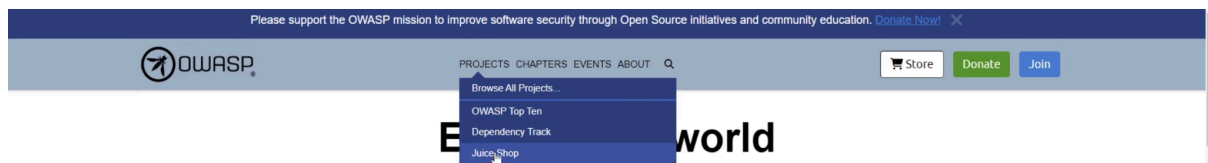
Así que nos vamos a ir a la máquina kali y vamos a comenzar a configurarlo.

Vamos a abrir nuestra consola y vamos a decirle sudo docker Vamos a hacer el pull primero para que nos lo descargue, si no lo copiamos y pegamos de la página, está bien, está haciendo la descarga bastante rápida.

```
sudo docker pull bkimminich/juice-shop
```

Como ya lo tendremos descargado en local, ahora ya lo siguiente que tendremos que hacer es decirle que se ejecute indicándole el puerto donde queremos que lo haga.

Y podríamos por ejemplo vamos a buscar dentro de OWASP para que veáis toda la documentación del proyecto de Juice Shop.



Aquí tenemos un poco cómo funciona, también tenemos vulnerabilidades que afectan a ese top 10, gente que lo ha hecho, cómo podemos instalarlo también el GitHub si queremos a nivel de instalaciones, cómo se haría al final nos va a llevar al GitHub y aquí podemos ver también cómo está organizado esa parte sencillita.

Vamos a volver a nuestra máquina virtual, lo tenemos, pues ahora lo que vamos a hacer va a ser es:

```
sudo docker run --rm -p 3000:3000 --name juice  
bkimminich/juice-shop
```

Yo le seguiría añadiendo el `--name juice` para ponerle un nombre como juice y le tenemos que decir de nuevo el nombre cual vamos a levantar esta vez ya lo tenemos descargado, así que `bkimminich/juice-shop`.

Tenemos todas las dependencias, revisa que tiene todo lo que necesita, que ya está descargado y nos dice OK, levantado en el puerto 3000.

Pues vamos a intentar acceder a ver si de verdad ese localhost en el puerto 3000 nos responde, en el navegador ponemos `localhost:3000`.

Y aquí estamos, este es Juice Shop, este es el aspecto que tiene la primera vez que nos conectamos nos da la bienvenida, nos dice es una aplicación web con un montón de vulnerabilidades que están creadas de manera voluntaria, se supone que es una de las mejores formas para desarrolladores para entender las vulnerabilidades y también para gente que está aprendiendo de ciberseguridad, temas de training, demostración, etc. Es un proyecto open source de OWASP y está desarrollado por voluntarios. El enlace para tener más información y le dices que OK si quieres ayuda para empezar o si quieres hacer un dismiss.

Vamos a decirle ayuda y vamos a ver qué nos va diciendo, nos va a ir guiando.

Esta aplicación tiene vulnerabilidades de seguridad, tu progreso se va a trackear en un scoreboard. Si no encuentras el link, tienes que buscar el scoreboard directamente como uno de los challenges.

Y aquí ya tendríamos una web aparentemente normal en la que hay una serie de productos que podemos comprar, un menú en el que podemos ver información de la web, tiene cuentas, que a día de hoy no tenemos cuenta, diferentes idiomas y estaría preparado para auditar como si fuese una auditoría real, 100 % en la vida real.

Tenemos un tercer entorno, este mucho más orientado a la vida real.

Aquí no vamos a poder probar vulnerabilidades en plan SQL Injection, está aquí, luego lo vamos a poder ir aprendiendo y la gracia es que podáis ir practicando lo que vais aprendiendo contra esta aplicación.

Es una buena forma de aprender.

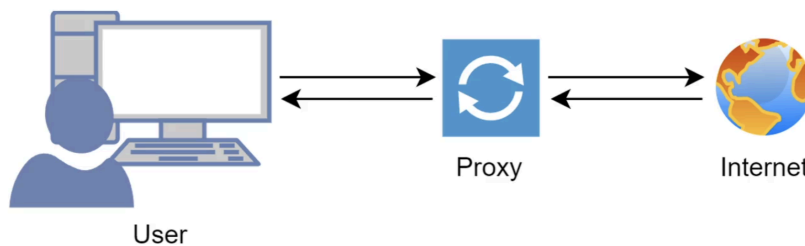
En este caso además con esa parte de que se parece a la vida real, que una vida real en una aplicación no te dice dónde hay vulnerabilidades, las tienes que encontrar.

Proxy Web: Burp

En esta sesión vamos a tratar el tema de los proxies, en concreto de los proxies web y vamos a ver qué es un proxy, cómo configurarlo para poder utilizarlo correctamente de cara a realizar auditorías de aplicaciones web.

Vamos a descubrir Foxy Proxy, que va a ser un gestor que nos permite administrar los proxies que tengamos en el navegador y también vamos a proponer el primero de los ejercicios relacionados con la configuración correcta del proxy para poder utilizarlo de cara a las futuras auditorías web.

¿Qué es un proxy?



Tenemos un servidor proxy, normalmente al final lo que es es un intermediario, o gateway, entre diferentes dispositivos de red.

Cuando estamos hablando de un proxy web, este intermediario va a actuar entre ese navegador y la aplicación web.

Y se va a colocar en medio, de manera que todas las solicitudes o peticiones que haga el usuario a la aplicación van a pasar primero por un proxy y es el proxy el que las va a redirigir a la aplicación web, que es la que está en Internet, como vemos aquí en las flechas.

Del usuario al proxy, del proxy a Internet, de Internet al proxy, del proxy al usuario.

Si el proxy solamente está en modo escucha, vamos a decir que solamente ve la información pasar de un lado a otro y no hace nada.

Lo que vamos a hacer con nuestro proxy web va a ser permitirnos modificar peticiones si es necesario, hacer pruebas y automatizar otro tipo de peticiones que nos permitan detectar vulnerabilidades.

Al final vamos a poder interceptarlas, vamos a poder modificarlas y eso nos va a dar una ventaja a la hora de descubrir vulnerabilidades que puedan existir en las aplicaciones web.

¿Para qué se suelen utilizar estos proxies webs en las organizaciones?

Aunque nosotros le vamos a dar un uso orientado a la auditoría web, estos proxies webs existen también en las organizaciones con otros fines, como por ejemplo controlar el uso que hacen los empleados de Internet.

Permite filtrar a qué páginas se puede acceder a qué páginas no, basándose en el tipo de páginas, las URLs, los horarios, grupos de usuario, etcétera. Permite controlar por dónde está navegando el usuario.

Y otra parte que también se suele utilizar es para la mejora del rendimiento, para reducir tiempos de respuesta, solicitudes usando una función de caché.

Son solamente dos ejemplos de usos de estos proxies para que veáis que no solamente se utilizan para esta parte de las auditorías web.

Pero ahora nosotros nos vamos a centrar en éstas dos funciones.

¿Qué es lo que vamos a hacer?

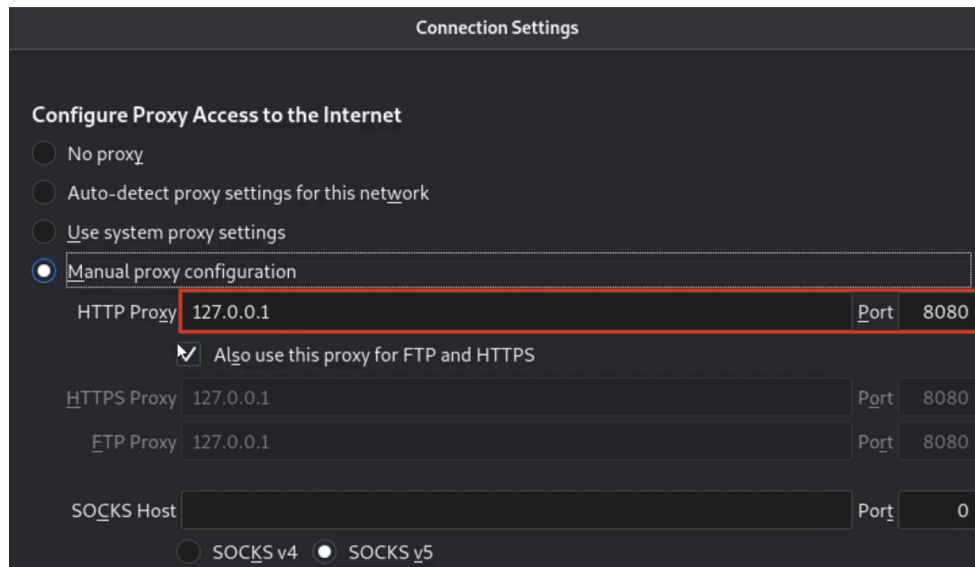
Nosotros vamos a utilizar Burp Suite como herramienta que va a actuar de proxy.

Existen otras como Zap, que es una herramienta de OWASP, pero la más completa, a pesar de las limitaciones que tiene la licencia gratuita que vamos a utilizar es Burp Suite Community Edition.

Configuración del navegador

Entonces lo que tenemos que hacer para poder configurar esta herramienta, configurar nuestro proxy, va a ser por un lado configurar el navegador y por otro lado la propia herramienta que viene ya configurada por defecto.

Al final lo que vamos a hacer, como vemos en la imagen de abajo, es decir que todo el tráfico que llega a nuestro navegador lo redirija a nuestra IP localhost, al puerto 8080 y nuestra herramienta lo que va a hacer va a estar en ese puerto 8080 escuchando, esperando a que llegue algo y cuando le llegue directamente es la que lo va a mandar a nuestro navegador, se va a poner ahí en medio.



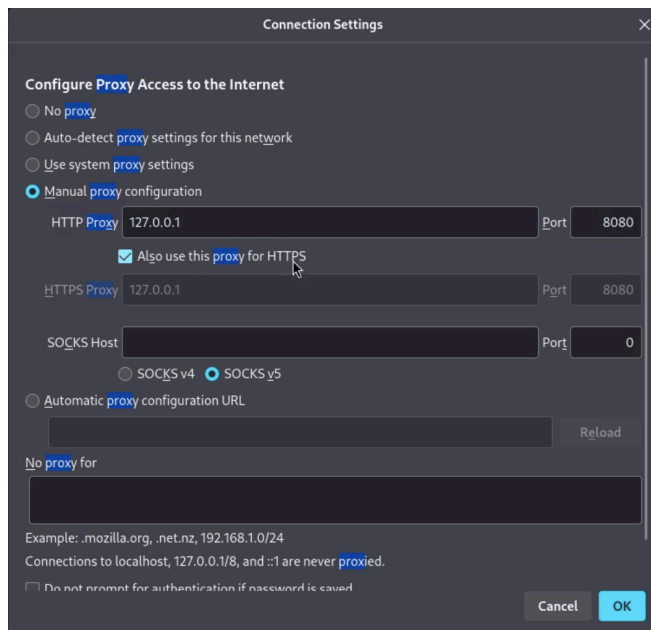
Cualquier petición que llegue desde Internet a nuestro navegador va a pasar por ese puerto 8080 y si solamente estamos mirando o si queremos que la función o la petición se ejecute, la pasaremos directamente al navegador. Estamos en medio.

Para ello necesitamos configurar primero el navegador.

Nos vamos a ir rápidamente a nuestro entorno de prueba, en kali, vamos a abrir nuestro navegador. Y nos tenemos que ir a las opciones, Settings.

Aquí podemos filtrar directamente como proxy para ir a la opción que queremos directamente y tenemos que configurar un proxy de forma manual.

Es aquí donde tenemos que poner el 127.0.0.1 y el puerto 8080, indicar que lo utilice tanto para HTTP como HTTPS y SOCKS si quisiéramos.

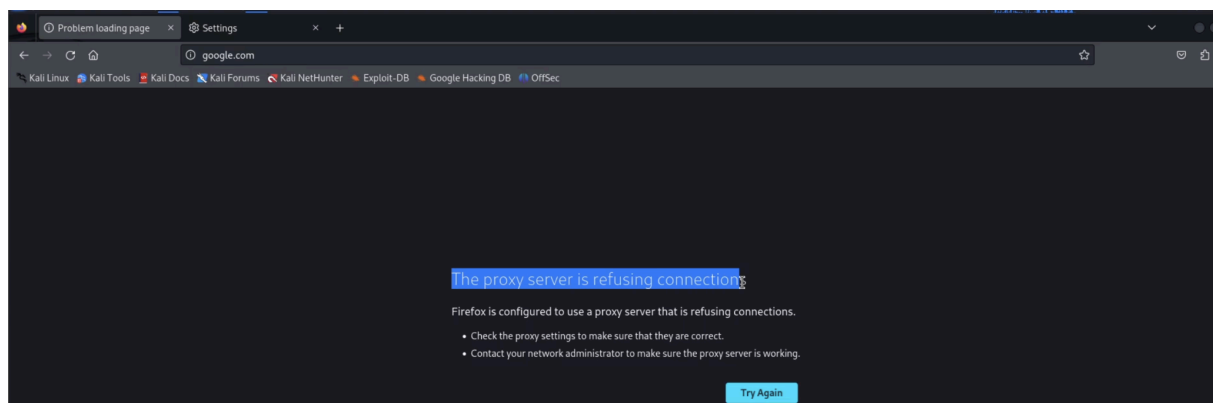


Vemos que no hay excepciones a nivel de proxy y le vamos a decir que OK.

A partir de este momento nuestro navegador va a redirigir las peticiones a ese puerto 8080.

¿Qué es lo que pasa?

Que si ahora nosotros intentamos acceder a Google, el proxy dice que está denegando las conexiones.



Realmente es que el proxy no sabe qué hacer con ellas, las está mandando al 8080 y no hay nadie que esté respondiendo.

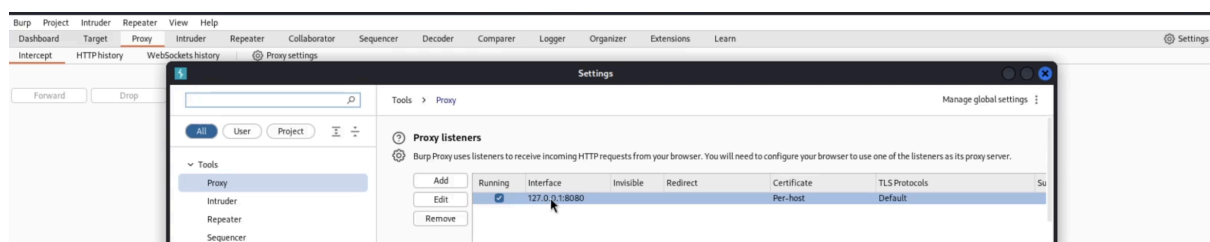
Configuración de Burp Suite

Lo siguiente que tenemos que hacer es abrir la aplicación de Burp Suite, la vamos a buscar porque viene instalada ya por defecto en Kali y vamos a levantar la aplicación Burp Suite Community Edition.

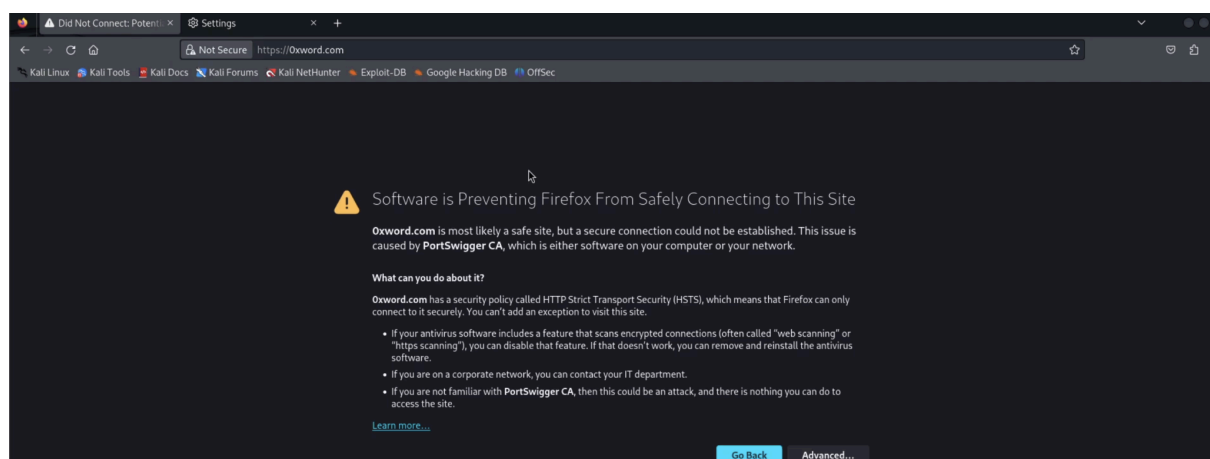
Esperar a que levante la primera vez va a tardar un poquito.

Aceptamos términos y condiciones, un proyecto temporal que es lo que nos permite hacer y levantamos Burp.

Cuando esté levantado, vamos a comprobar que en la pestaña Proxy, en Proxy Settings, tenemos configurado un listener por defecto que está en ese 127.0.0.1 8080.

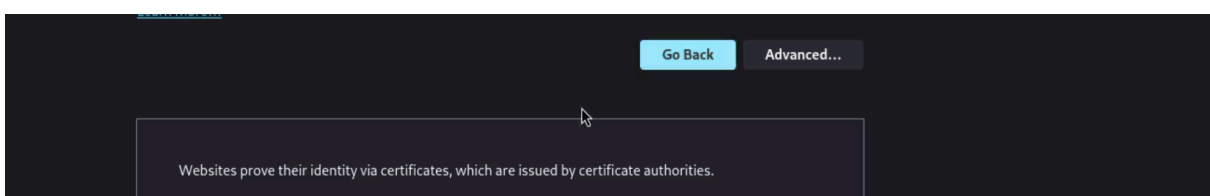


De manera que si ahora por ejemplo, vamos a intentar acceder a <http://0xword.com>



Ahora hemos cambiado de error, ya no me dice que no sabe qué hacer, me está diciendo que me está previniendo de acceder a un sitio porque no conoce este certificado.

Hay un PortSwigger CA que no es el que espera, me avisa, le doy a advanced.

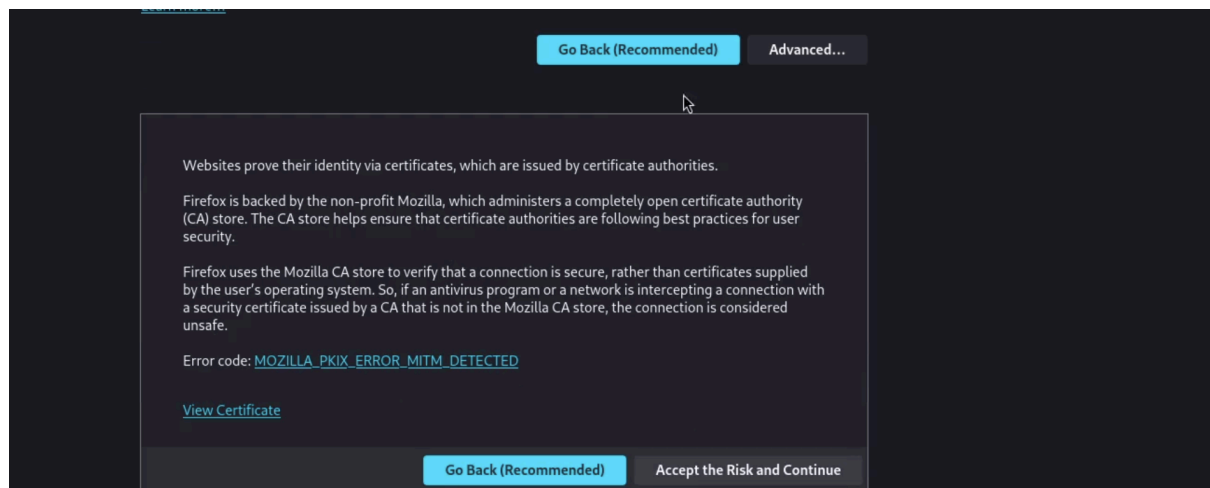


Incluso así no me deja hacer nada porque tiene una configuración de ciberseguridad, la página tiene la cabecera HSTS y no me deja conectarme.

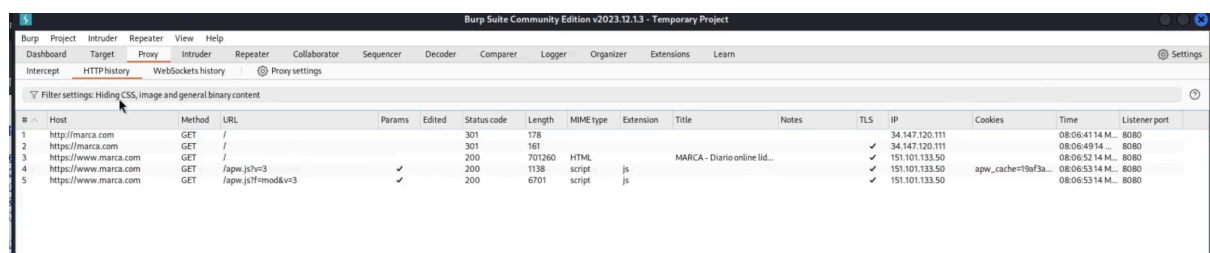
Esto es que la página tiene un nivel de seguridad correcto, pero de momento esto parece que ya nos deja acceder, que ya está haciendo algo.

Si nos vamos a nuestro Burp, tenemos aquí un historial que de momento no ha cogido nada, un intercept que tampoco.

Vamos a probar una página que no tenga estas medidas de seguridad, por ejemplo un periódico deportivo, el marca.com, fijaros, me da el mismo error pero aquí abajo me deja aceptar el riesgo y continuar.



Cuando yo le dé aceptar la página se me debería de cargar de aquella manera y nuestro servidor proxy ya está capturando las peticiones.



Voy a cerrar ya esta página de el periódico deportivo y tendríamos aquí las peticiones que ha ido capturando.

Vamos a limpiar esto, limpiamos el historial y nos queda de nuevo la herramienta limpia.

Lo siguiente que decíamos era el burp, que vemos que por defecto tiene este listener preparado y activo y con esto lo teníamos configurado.

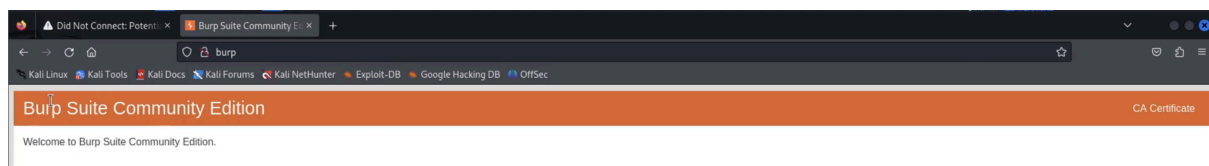
No nos ha dejado acceder a esa página HTTPs como era 0xword porque tenía algo configurado que es la cabecera HSTS.

¿Qué es lo que está pasando? Nos da un error de certificado.

Vamos a volver a la página y vamos a corregirlo. Esto es parte de la configuración que tendremos que hacer al principio.

Vemos aquí que nos dice que el certificado es de PortSwigger.

Ahora que tengo el error y que tengo funcionando la herramienta y configurado el navegador tengo que acceder a `http://burp`



Y me lleva a esta página que me dice Bienvenido al Burp Suite Community Edition y aquí a la derecha me aparece el certificado.

Vamos a descargar este certificado que se me ha descargado en Descargas y lo que vamos a hacer ahora va a ser instalarlo.

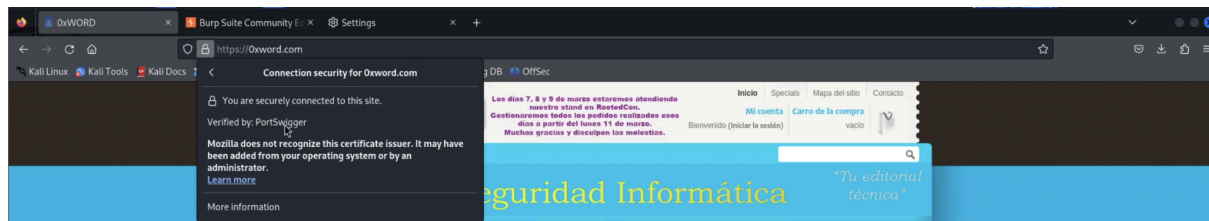
Para ello en los settings del navegador, filtro por certificates me voy a ir a View Certificates y vamos a importar el certificado que me he descargado.

Nos aparece este pop up si quieres instalar de verdad este certificado, esta nueva CA (Certificate Authority) que si confiamos en ella le vamos a decir que sí, tanto para identificar website como para identificar emails, le vamos a decir que OK.

Y a partir de ese momento tendremos un certificado nuevo instalado que es el de PortSwigger, ya me lo he pasado, aquí está, ya está instalado.

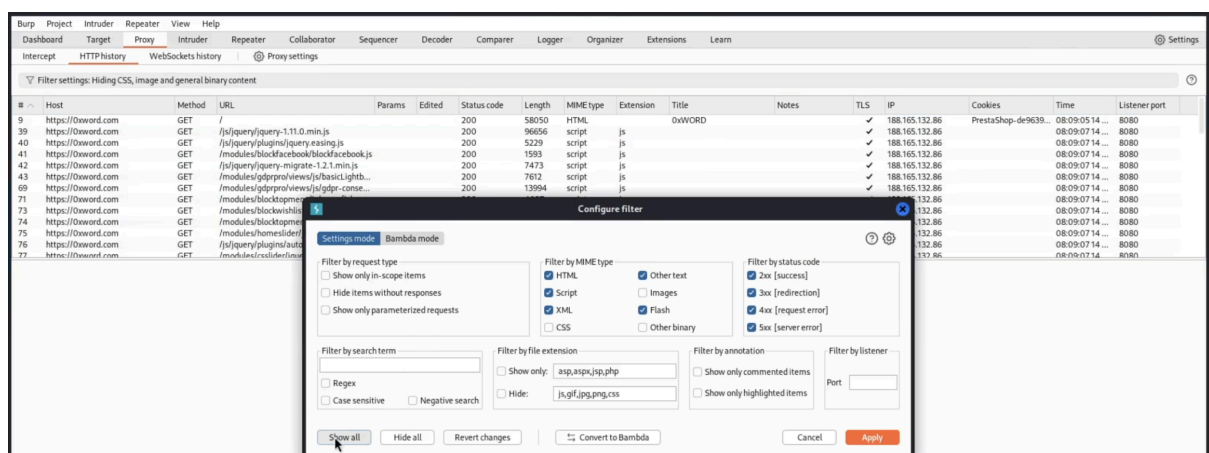
Vamos a ver qué es lo que pasa ahora cuando volvemos a cargar 0xword, recordad, tenemos esta página del histórico vacía, vamos a volver a cargar 0xword.

Ahora fijaros aquí ya el not secure ha desaparecido del navegador, aquí ya nos está cargando la página y si nos vamos a ver el certificado nos dice que es una conexión segura, pero que el certificado es el de PortSwigger.



Como ya confiamos en él y lo hemos metido en lista blanca o whitelist, ya podemos cargar páginas a través de HTTPs y lo más importante, todas las peticiones que hace esta página, vamos a poner que nos enseñe todas en los filtros, todas las peticiones que se han hecho para cargar 0xword, fijaros que llaman a temas, a módulos, cualquier imagen, cualquier librería se está cargando aquí.

Tenemos un total de 237 peticiones, 237 más bien que se han realizado para cargar la página.



¿Y por qué quito esas?

Porque las primeras que habíamos hecho de marca no cuentan, empezamos a contar en la 9, así con todas estas peticiones que ya estamos capturando tenemos aquí un proxy funcionando.

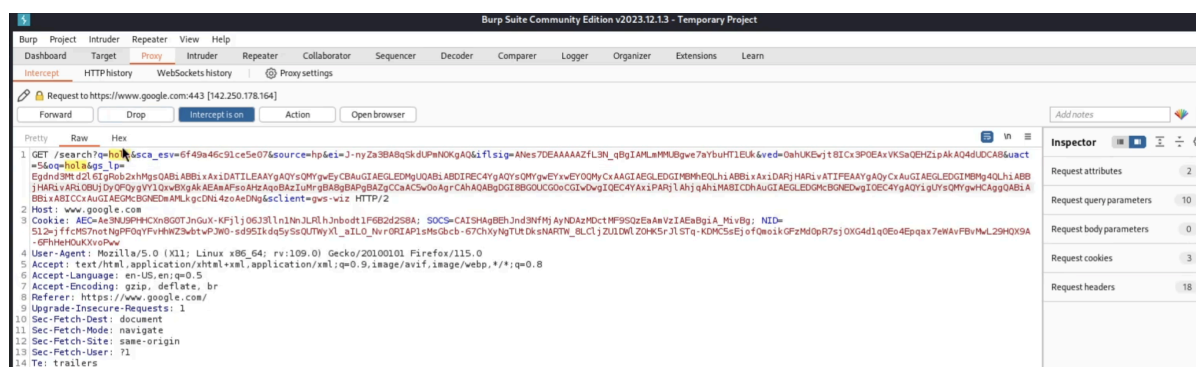
Vamos a ver por ejemplo otra página que tiene la cabecera HSTS configurada, pues es el propio Google.

Intercept con Burp Suite

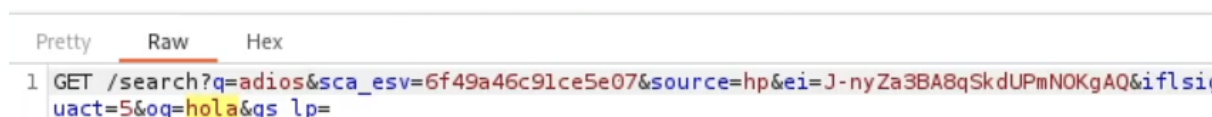
Aquí ¿qué es lo que nos va a permitir hacer burp?

Pues por ejemplo interceptar peticiones.

Vamos a poner hola en el navegador de google, voy a interceptar clicando a Intercept y voy a buscar. Iremos clicando a forward para buscar una en la que aparezca aquí una query que ponga hola, . Vemos que google está pensando, está buscando hola en google.



Ahora mismo esta petición está parada, Google vemos que está pensando, si yo modifico aquí este hola y vamos a poner adios, vamos a ver qué es lo que pasa.



Le puedo dar a forward para que continúe con este cambio que yo he hecho. Vamos a dar a forward, quito el intercept y fijaros que el resultado que me ha dado por atrás google es adios, me ha buscado adios finalmente, porque hemos interceptado la búsqueda y la hemos modificado.

Eso me permite parar peticiones, modificar on the fly y cambiar el funcionamiento de la aplicación.

Si le damos algo que no espera puede que cambie el comportamiento y lo estamos parando en medio.

A lo mejor podemos meter un string que no es posible meter directamente desde teclado en la web, o sea, no es posible meterlo desde el lado cliente, pero puede

llegar al servidor y ver cómo responde. Por ejemplo, un campo fecha, imaginaos que tenéis que poner del 01 del 01 de 2024, pues está limitado a números, pero a lo mejor sólo en el navegador, si yo lo intercepto y de repente pongo mi nombre, ¿Qué es lo que pasa? ¿Cómo va a responder el servidor?

Y eso son partes de las pruebas de seguridad que tenemos que ir haciendo.

Foxy Proxy

Así que tenemos el certificado ya configurado, funciona el HTTPs y ahora os voy a hablar de Foxy Proxy muy rápido.

Foxy Proxy es un add-on de navegador, es una extensión, la podemos instalar tanto en Firefox como en Chrome y lo que nos va a hacer es permitir configurar este proxy para poderlo activar y desactivar con un solo clic.

¿Y por qué esto es fácil o por qué esto es interesante?

Porque tener que meternos en las settings del navegador, buscar los proxies, quitarlos o ponerlos, puede llegar a ser cansado.

Así que vamos a intentar hacerlo con un clic para que sea lo más cómodo posible para nosotros como auditores.

Vamos a hacer una cosa antes que nada, voy a hacer en settings, antes de nada voy a deshabilitar el proxy, en lugar del proxy manual que utiliza el de sistema. Es decir, que yo ahora cuando vaya a la página de Foxy Proxy y me vaya a Burp, veré que no está capturando peticiones nuevas.

Estamos aquí en la última, estamos aquí en Google y yo ahora voy a irme a Foxy Proxy y aquí seguimos en la 401 no estamos capturando peticiones.

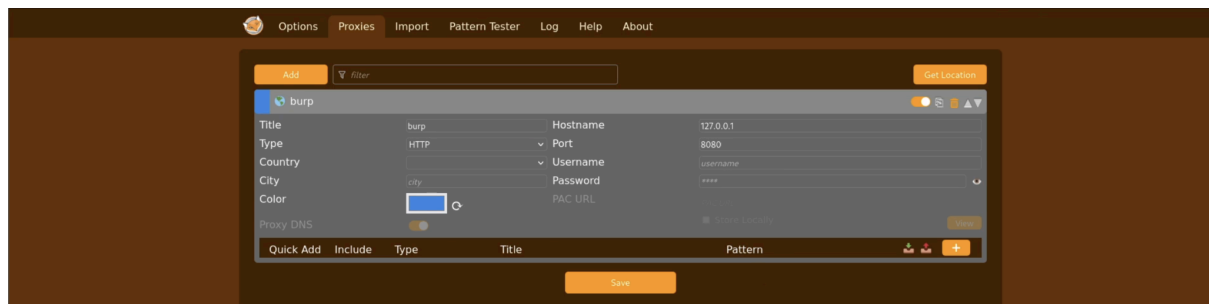
Este add-on lo vamos a agregar a Firefox, vamos a decir que nos lo añada, que lo permita en ventanas privadas. Y ahora ya tenemos aquí un add-on que vamos a meter dentro del menú.

¿Cómo funciona? Haremos clic, nos dice que está deshabilitado, vámonos a las Options, y dentro de las opciones le vamos a decir que en proxies queremos añadir

uno, y a este proxy le vamos a llamar Burp y vamos a rellenarlo de la misma manera que teníamos antes.

El hostname será la 127.0.0.1 y el puerto que nos interesa es el 8080.

Le podemos poner un color para que sea identificativo, pues por ejemplo ahora que tiene el burp el colorcito azulito, le ponemos un color azul. Le damos a seleccionar, le damos a guardar y nos podemos olvidar de esta parte.



Ya lo hemos configurado pero todavía no está activo.

¿Cómo vamos a activarlo?

Yo aquí ahora solamente hago clic sobre el icono que es el zorrillo y ya donde me aparece el disable, que es lo que está activa, me aparece también el nombre del proxy que acabamos de configurar.

Pues vamos a decirle que nos lo active.

Lo activamos, aquí arriba se pone, no se verá porque a lo mejor es muy pequeñito, pone burp en color azul y a partir de ese momento está funcionando el proxy.

De manera que si yo aquí ahora vuelvo a irme a 0xword, aquí ya está capturando, fijaros todas las peticiones que va capturando.

Ahora por ejemplo podemos activar y desactivar el proxy cuando queramos de forma fácil y rápida.

Imaginemos qué estamos haciendo de la auditoría, perfecto, lo tenemos activo. Oye, que necesitamos hacer una búsqueda en Internet y no queremos que nos ensucie esto, Pues igual, hacemos clic deshabilitar, hacemos clic habilitar.

Entonces es una manera muy fácil de gestionar nuestros proxies porque también puede que necesitemos varios.

Ejercicio

El siguiente ejercicio simplemente es la configuración del proxy.

Exercise: Proxy configuration

- Configure Burp in your environment
- Install Foxy Proxy
- Capture Google traffic with Burp

Es volver a repetir lo que he explicado, comprobar que tenemos configurado el navegador o descargarnos Foxy Proxy y configurarlo.

Una vez que lo tengamos instalado también FoxyProxy, vamos a intentar capturar tráfico de Google.

Cuando tengamos capturando tráfico de Google que nos aparezca en el historial, quiere decir que tenemos nuestra herramienta Burp lista para usar y por lo tanto ya no vamos a necesitar hacer configuraciones de este tipo a lo largo del módulo.

Tendríamos nuestra herramienta funcionando y tanto Burp como los proxies correctamente configurados.

Así que ese es el ejercicio realmente es solamente volver a repetir lo que hemos visto en clase para que tengáis el entorno preparado.

A nivel de conclusión, hemos visto qué es un proxy, qué es en concreto un proxy web y cómo actúa de intermediario entre nuestro navegador e Internet, la aplicación que vayamos a auditar.

Tenemos preparada para ser utilizada la herramienta que va a ser la herramienta principal que se utiliza en las auditorías de aplicaciones web porque nos va a

facilitar mucho el trabajo y nos va a permitir identificar vulnerabilidades de una manera más sencilla, además de lanzar otro tipo de ataques de una manera automatizada.

Y tenemos nuestro Burp configurado.

Vamos a tratar el tema de los proxies y vamos a ver en detalle en qué consiste Burp Proxy y todas sus funcionalidades, ya que es una herramienta que es un poco como una navaja suiza y tiene diferentes funcionalidades en una, además, por supuesto, de ser el proxy web que vamos a utilizar y que sería su funcionalidad principal.

Para entender por qué es la herramienta principal que necesitamos conocer y tener configurada de cara a realizar auditorías web, es necesario entender qué es lo que nos puede aportar, qué es lo que podemos hacer con ella y cómo funcionan todas sus sub herramientas.

Funcionalidad: Proxy

Comenzamos con la parte del proxy, que es la funcionalidad core y nos vamos a ir a la máquina virtual atacante de kali para que lo vayamos viendo.

Aquí voy a levantar burp y voy a levantar DVWA vía docker en la terminal. Vamos al navegador, en Activamos Burp en FoxyProxy y accedemos a localhost para entrar a la aplicación web de DVWA

Y aquí ya en history de Burp nos va a aparecer.

Otra cosa que no he comentado es que además de acceder por localhost, podríamos acceder directamente por nuestra IP.

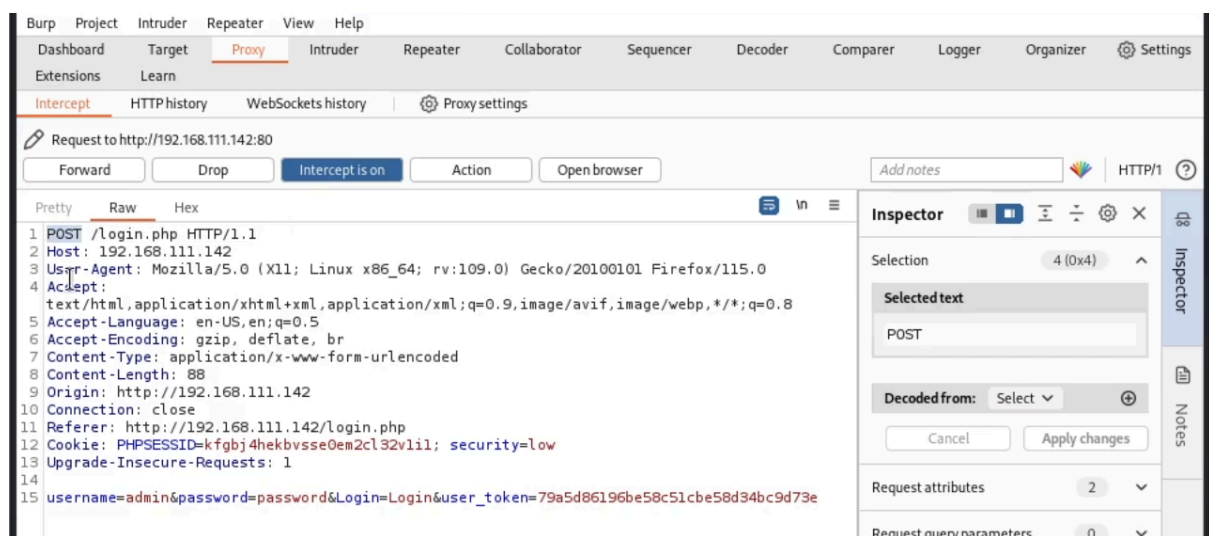
Entonces en mi caso sería por la 192.168.111.142 y podríamos acceder de la misma manera.

Aquí ya lo tenemos de las dos maneras, en burp vemos las dos maneras de acceder. Vamos a utilizar la de la IP, que creo que quizás es más fácil de ver, así que está en mayúsculas.

En DVWA vamos a acceder con el user el password, vamos a poner admin password y vemos que aquí en burp vamos generando peticiones, que nos lo recuerde y ya tendríamos la aplicación aquí abierta.

Esta parte del proxy ¿Que nos permite?

Pues nos permite interceptar peticiones, por ejemplo, nos ponemos a interceptar dándole en burp a Intercept is on y vamos a iniciar sesión con admin y con password. Hacemos clic en login y aquí tenemos parada la petición, es decir, no entra hasta que en burp no le demos a forward.

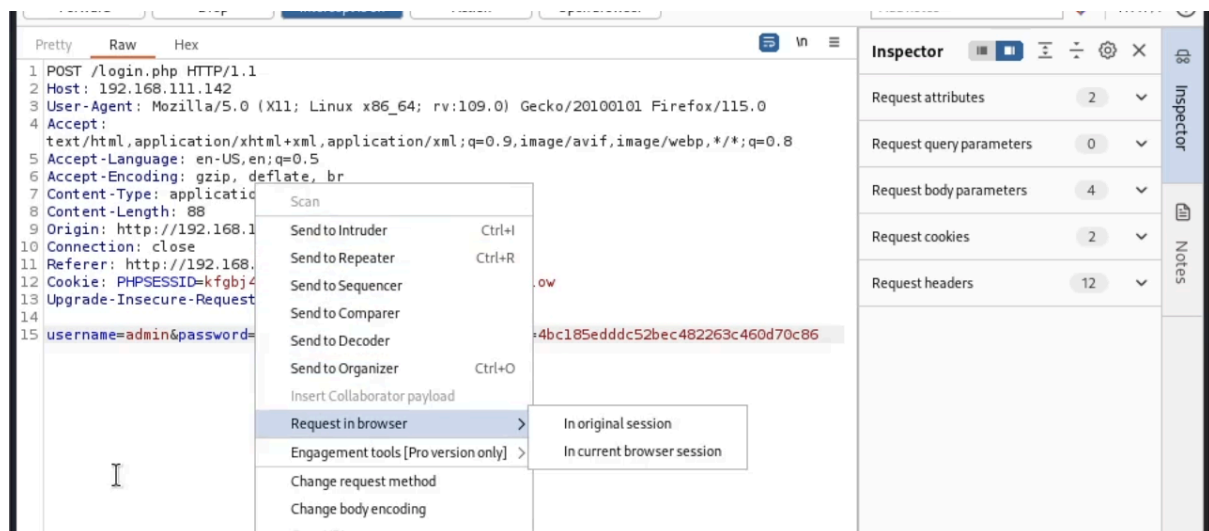


Vemos en burp que la aplicación es tipo post, las cabeceras, aquí podríamos ver de momento la cookie que tenemos, el user agent que es nuestro navegador, a que aquí no nos está dando gran información y estamos viendo cómo envía el username y la contraseña en la última línea, tiene también un token de usuario y vemos el contenido.

Aquí podemos o bien dejar que pase la petición, que sería darle a forward, o bien ignorarla, que sería darle a drop.

Vamos a darle a drop, y nos da error en DVWA, no hemos entrado.

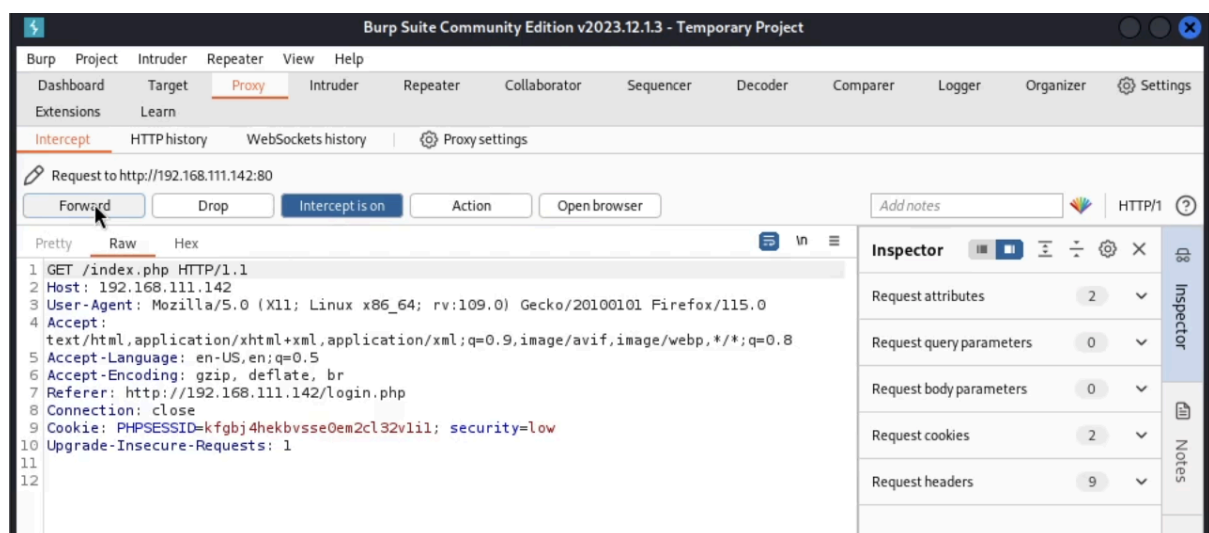
Volvemos a hacerlo, ponemos el admin / password, volvemos a interceptar, le doy a login. Si, a veces se nos queda esto pensando y estamos viendo no me sale en history, revisad que no esté parado en intercept porque a veces creemos que no está funcionando y en realidad lo que ha hecho ya es interceptar la petición.



Aquí, antes de darle a forward, podríamos modificar la petición, hacer botón derecho y enviarla a cualquiera de las otras herramientas, pero de momento lo vamos a dejar así.

Vamos a darle a forward, que sería lo mismo que quitar el intercept si no lo vamos a necesitar más, así que quito el intercept.

Bueno, damos a forward para que veáis cómo funciona.



Me saldría la siguiente funcionalidad, la siguiente petición.

Aquí ya me ha generado mi sesión de usuario, mi cookie de sesión y el nivel de seguridad.

Vemos que está puesto a través de la cookie.

Le doy a forward y ahora sí estamos dentro, o sea que el login lleva dos peticiones, primero usuario y contraseña y luego una petición en la que nos devuelve o en la que enviamos ya nuestra cookie de sesión con el nivel de dificultad que tiene configurada la máquina.

Aquí quitamos ya el intercept y sería un poco cómo funciona.

Esta parte del proxy nos va a servir para ver internamente cómo está mandando las peticiones, cómo de grande es una petición, etc.

The screenshot shows the Burp Suite interface. The top menu bar includes Burp, Project, Intruder, Repeater, View, and Help. Below it is a toolbar with various tools like Dashboard, Target, Proxy, Intruder, Repeater, Collaborator, Sequencer, Decoder, Comparer, Logger, Organizer, and Settings. The main window is divided into several panes. The top pane shows the HTTP history table with columns for #, Host, Method, URL, Params, Edited, Status code, Length, MIME type, Extension, and Title. The table lists several requests, with the last one (request 19) highlighted. The bottom pane is split into two sections: 'Request' and 'Response'. The 'Request' section shows a POST request to /login.php with various headers and a body containing login credentials. The 'Response' section shows the server's response, which is an HTTP 302 Found status, indicating a redirect. The 'Inspector' pane on the right shows the details of the selected request, including request attributes, body parameters, cookies, headers, and response headers.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title
4	http://192.168.111.142	GET	/			302	442	HTML		
5	http://192.168.111.142	GET	/login.php			200	1814	HTML	php	Login:: Darr
9	http://192.168.111.142	POST	/login.php		✓	302	300	HTML	php	
10	http://192.168.111.142	GET	/index.php			200	7099	HTML	php	Welcome ::
12	http://192.168.111.142	GET	/dwa/js/add_event_listeners.js			200	882	script	js	
13	http://192.168.111.142	GET	/dwa/js/dwaPage.js			200	1320	script	js	
15	http://192.168.111.142	GET	/logout.php			302	300	HTML	php	
16	http://192.168.111.142	GET	/login.php			200	1860	HTML	php	Login:: Darr
17	http://192.168.111.142	POST	/login.php		✓			HTML	php	
18	http://192.168.111.142	GET	/login.php			200	1814	HTML	php	Login:: Darr
19	http://192.168.111.142	POST	/login.php		✓	302	300	HTML	php	
20	http://192.168.111.142	GET	/index.php			200	7099	HTML	php	Welcome ::

Request

1 POST /login.php HTTP/1.1

2 Host: 192.168.111.142

3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8

5 Accept-Language: en-US,en;q=0.5

6 Accept-Encoding: gzip, deflate, br

7 Content-Type: application/x-www-form-urlencoded

8 Content-Length: 88

9 Origin: http://192.168.111.142

10 Connection: close

11 Referer: http://192.168.111.142/login.php

12 Cookie: PHPSESSID=kfgbj4hekbvsse0em2cl32v1l1; security=low

13 Upgrade-Insecure-Requests: 1

14

15 username=admin&password=password&Login=Login&user_token=4bc185edddc52bec482263c460d70c86

Response

1 HTTP/1.1 302 Found

2 Date: Thu, 14 Mar 2024 12:50:12 GMT

3 Server: Apache/2.4.25 (Debian)

4 Expires: Thu, 19 Nov 1981 08:52:00 GMT

5 Cache-Control: no-store, no-cache, must-revalidate

6 Pragma: no-cache

7 Location: index.php

8 Content-Length: 0

9 Connection: close

10 Content-Type: text/html; charset=UTF-8

11

12

Inspector

Request attributes 2

Request body parameters 4

Request cookies 2

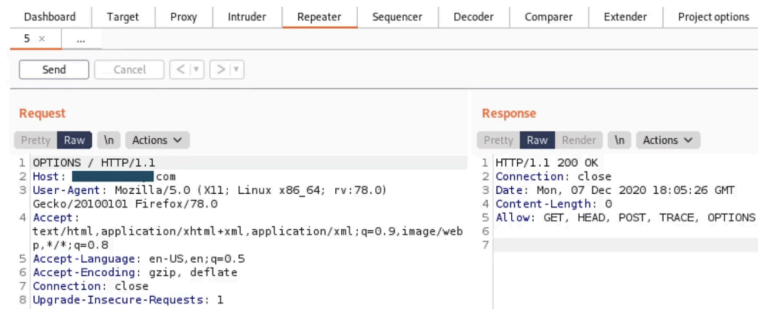
Request headers 12

Response headers 9

Si quisiéramos ver también la respuesta, vámonos aquí a este login y vemos la respuesta cuando le hemos mandado nuestro admin password, vemos que aquí nos devuelve cabeceras de respuesta, en la cual vemos el servidor que hay detrás, que es un Apache 2.4.25, y en principio aquí no nos da más información, si hubiese cabeceras de seguridad también irían aquí incluidas.

Esta sería la parte del proxy, que es la funcionalidad principal.

Funcionalidad: Repeater



Pasamos a la siguiente funcionalidad, el repeater.

El **Repeater** de **Burp Suite** es una herramienta esencial para realizar pruebas manuales en aplicaciones web, permitiéndote manipular y reenviar peticiones **HTTP/HTTPS**.

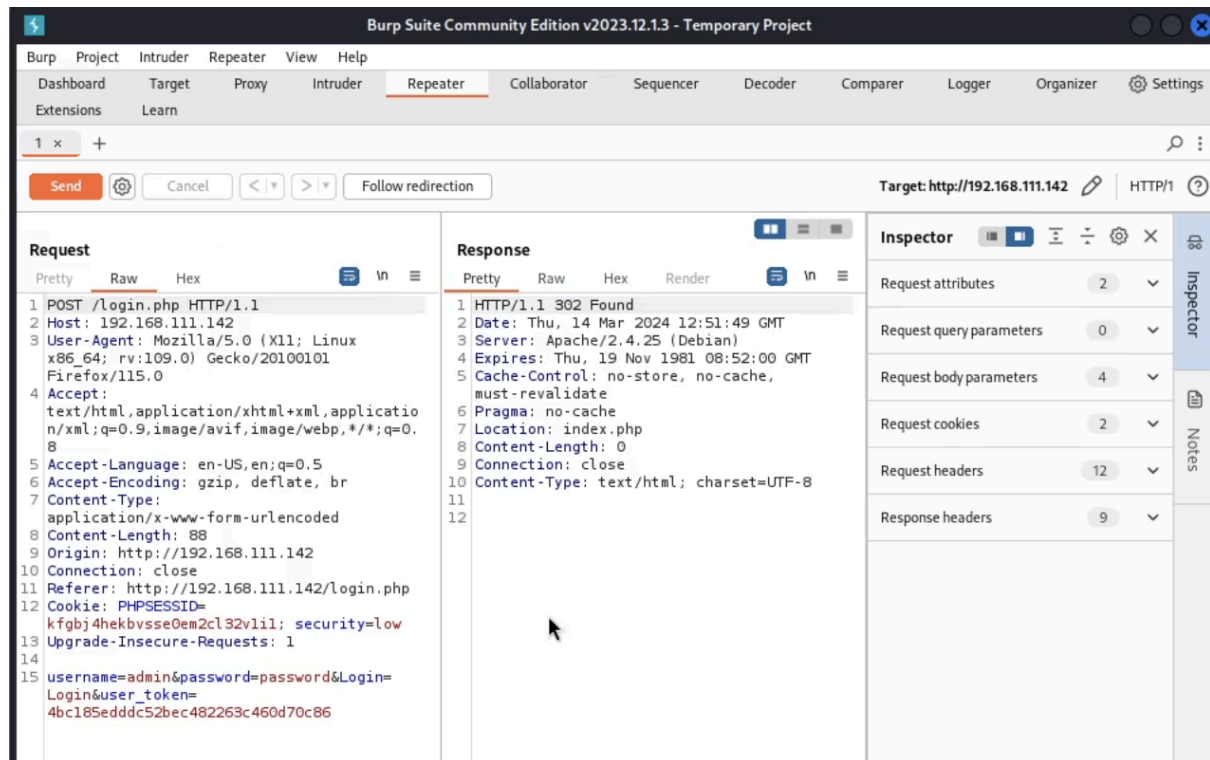
Su función principal es capturar una petición web, modificarla y enviarla al servidor tantas veces como necesites. Por ejemplo, puedes cambiar valores de parámetros, encabezados, o el cuerpo de la petición para probar diferentes escenarios de ataque. Esto es muy útil en la fase de análisis de vulnerabilidades, ya que te permite aislar un fallo y ver cómo reacciona el servidor ante entradas inesperadas.

En resumen, el Repeater sirve para:

- **Modificar peticiones:** Puedes editar cualquier parte de una petición antes de reenviarla.
- **Probar vulnerabilidades:** Es ideal para probar vulnerabilidades como la inyección SQL o XSS.
- **Analizar respuestas:** Te permite comparar las respuestas del servidor después de cada modificación para entender su comportamiento.

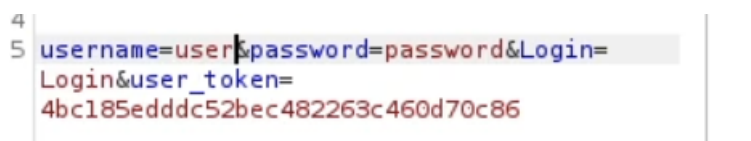
El repeater lo que va a hacer es permitirnos mandar peticiones unitarias al servidor de manera transparente al navegador para permitir analizar tanto la petición como la respuesta y cómo se comporta la aplicación.

Entonces aquí lo que tendríamos que hacer, imaginaros, volvemos aquí, esta misma petición del admin password la mandamos al repeater con botón derecho y Send to repeater, y vamos a la pestaña del repeater.

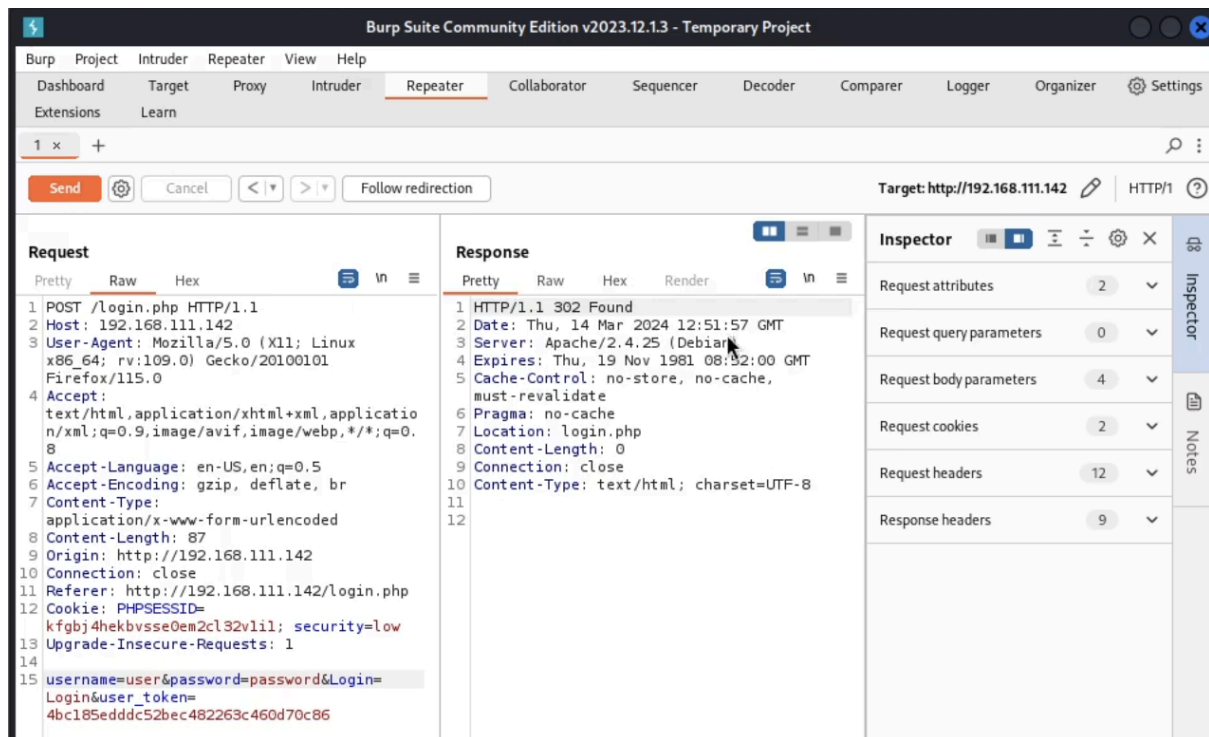


De base, solamente tenemos la petición, si yo hago de nuevo el send me da un 302 found, pero no me da más información.

Vamos a imaginar que aquí ponemos usuario user y hacemos el send.



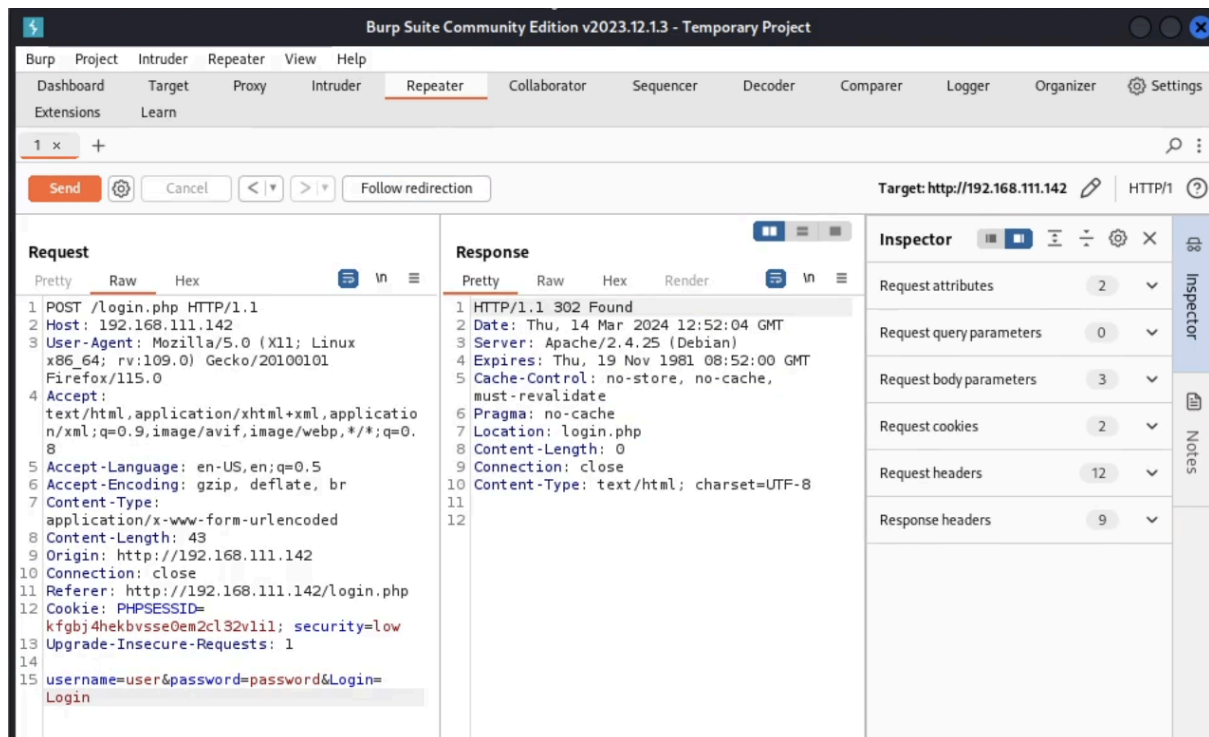
Esto no me está dando información, yo ya tengo una sesión iniciada:



Si quitamos por ejemplo el user_token y le damos a send:

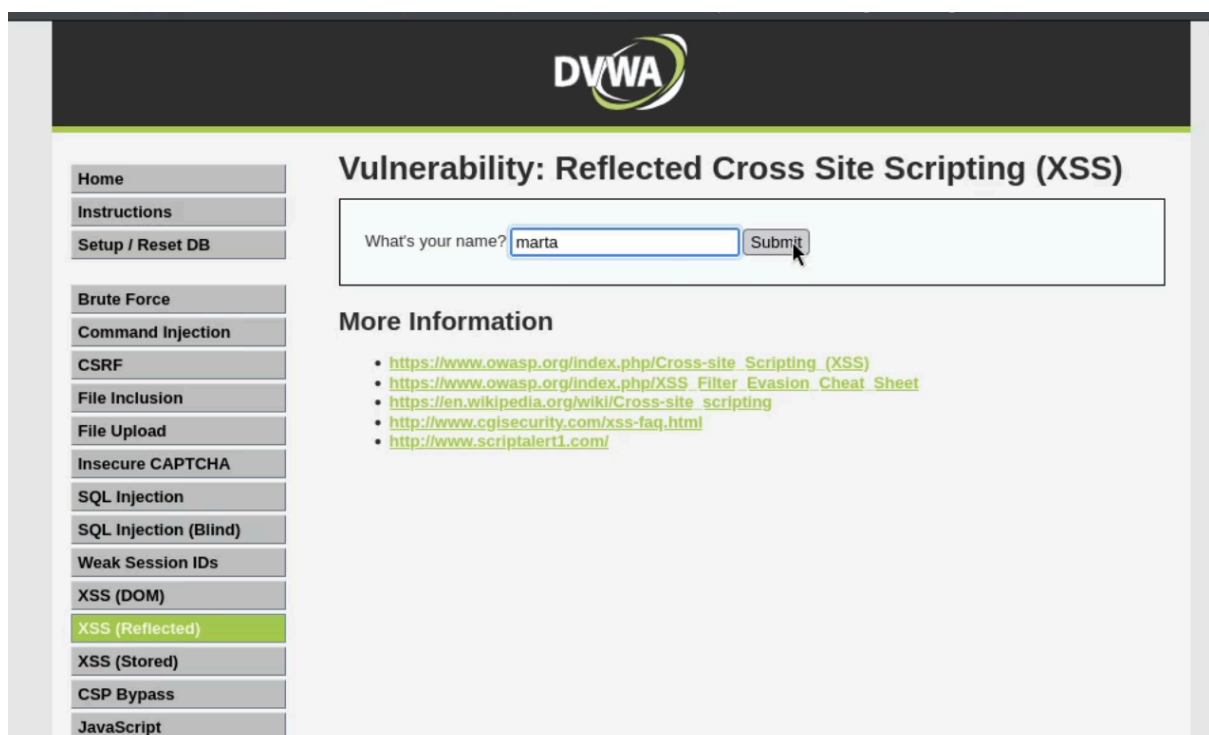
```
4  
5 username=user&password=password&Login=  
Login
```

302 found quizás no es la petición con el mejor ejemplo en esta parte del repeater.

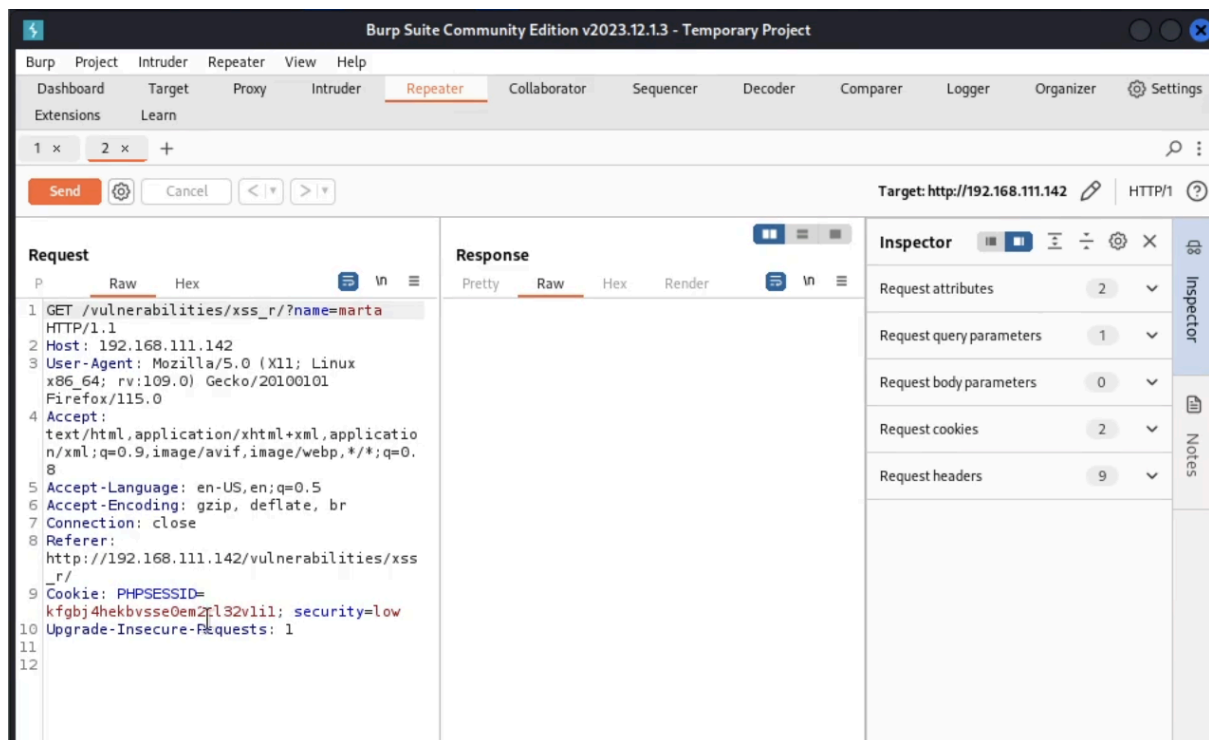


Cambiamos de petición para ésta parte del repeater.

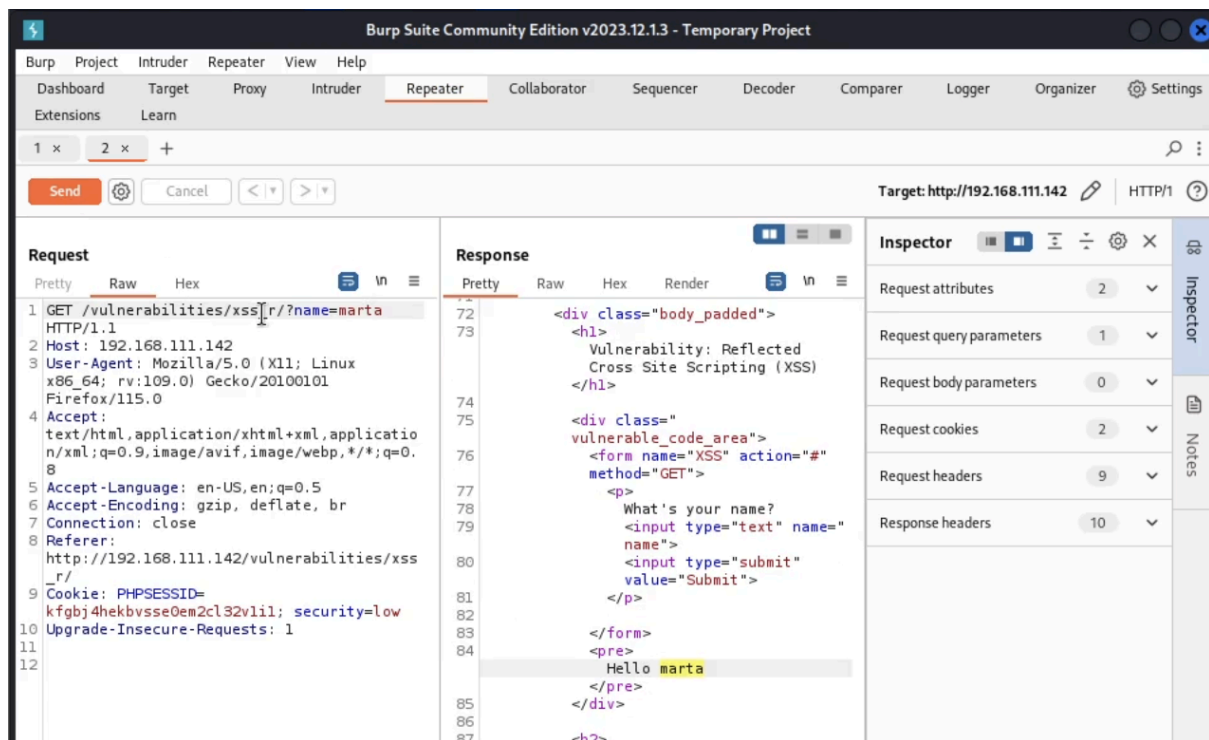
Vamos a probar por ejemplo un Cross Site scripting reflejado y aquí ponemos marta submit, y me dice hello marta.



Pues nos vamos a burp y sería esta la que mandamos al repeater, la seleccionamos en el history, botón derecho y Send to repeater.



Le doy a send para más información en la Response:

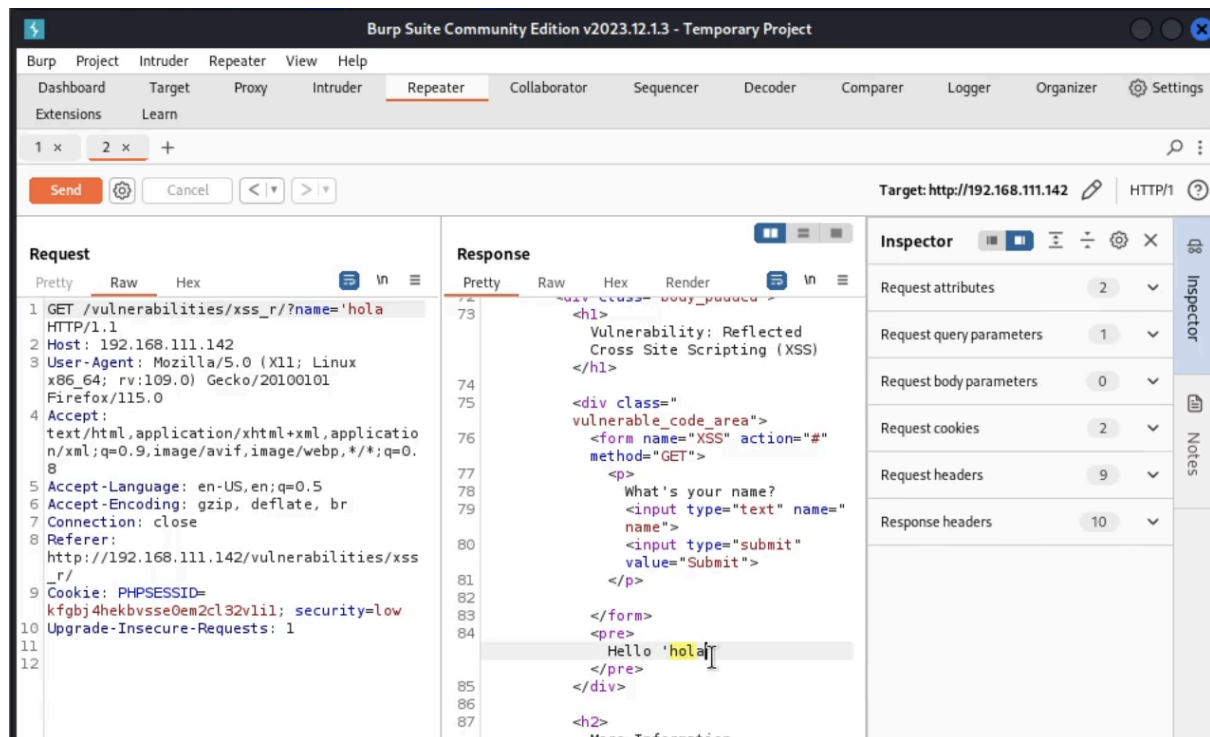


Y aquí es donde jugaríamos en plan ¿Qué está mandando?

Vale, si mando marta aquí me saldrá, el Hello Marta. Hemos buscado marta abajo para encontrar matches.

¿Qué es lo que pasa si en la request cambiamos el hello marta, y en lugar de marta escribo 'hola' ?

Le damos a send y ahora vamos a buscar aquí el 'hola, y aquí hay un hello comilla hola.



Esto lo podíamos ver directamente en el navegador, pero a veces va a ser necesario mandar las peticiones al repeater para hacerlo de una manera más sencilla a la hora de analizar estas peticiones.

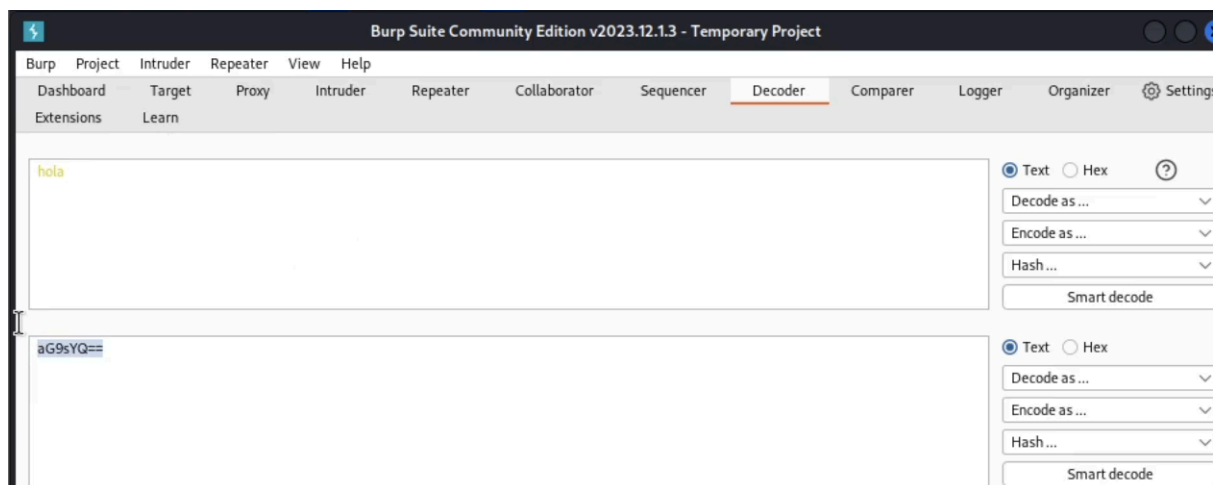
Estas peticiones de aquí no se incluyen en lo que sería el historial, las está mandando el burp, pero no se incluyen dentro del historial y se mandan directamente al servidor, no pasan por el navegador. Es interesante que lo tengamos claro.

Funcionalidad: Decoder

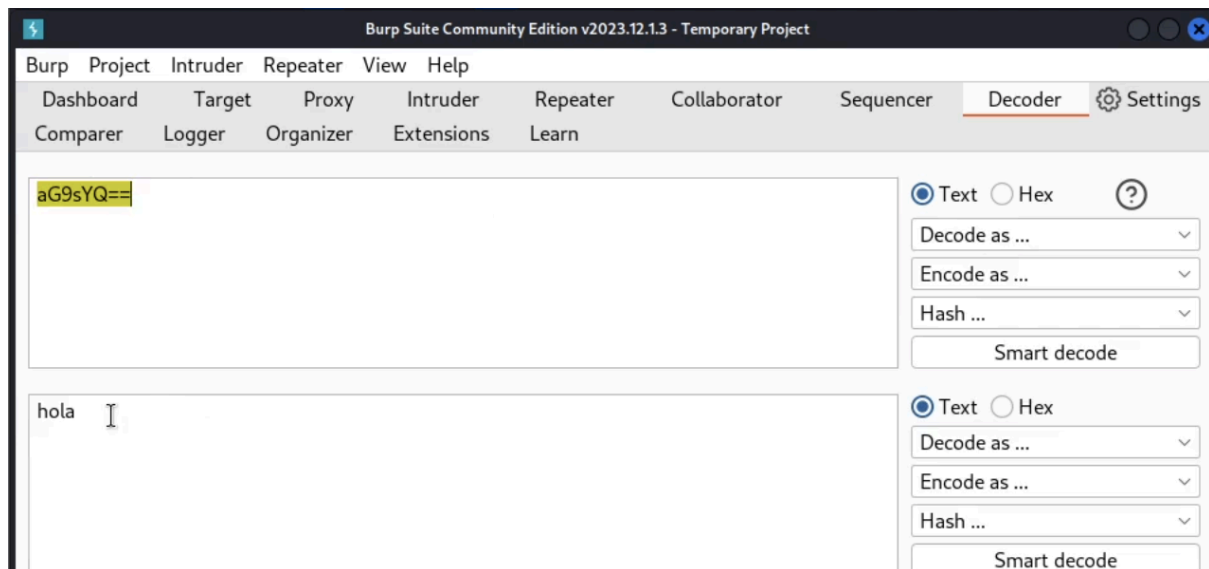
Decoder es una pequeña herramienta que nos va a permitir decodificar pequeños códigos o pequeñas codificaciones de las más comunes cuando nos encontremos strings codificadas, lo más común suele ser que lo encontremos en tokens de sesión, en cookies, en partes de la URL, lo vamos a poder decodificar directamente desde la aplicación, tanto codificar como decodificar.

Nos iríamos al decoder y aquí podemos poner lo que queramos.

Imaginaros que vamos a poner hola y lo que vamos a hacer es primero encodearlo en base 64 y tenemos esta captura.



Si yo me encuentro esta cadena en algún punto de la web, al ver ya el "==" puedo pensar que es base 64, me vendría aquí y le diría mira me encontré esta cadena, quiero que me la decodifiques como base 64 y me sale hola.

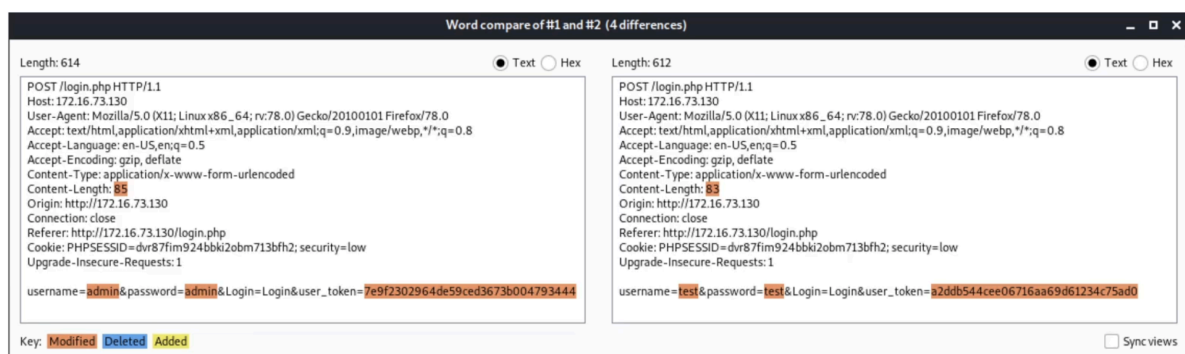


Imaginaros que esto es una cookie y en lugar de hola es en nombre del usuario o incluso la contraseña, no debería ser ninguna de las dos, pero algunas veces nos lo hemos encontrado, así que sería para lo que sirve esta herramienta del decoder, codificar y decodificar algunos tipos de codificaciones que nos vamos a encontrar URL, HTML, base 64, ASCII, hexadecimal, octal, binario y gzip. Y a la hora de encodear lo mismo.

También puede haber algún tipo de hashes y aquí te los más comunes que puede ser capaz de darles la vuelta.

Yo normalmente lo utilizo para decodificar y para codificar.

Funcionalidad: Comparer



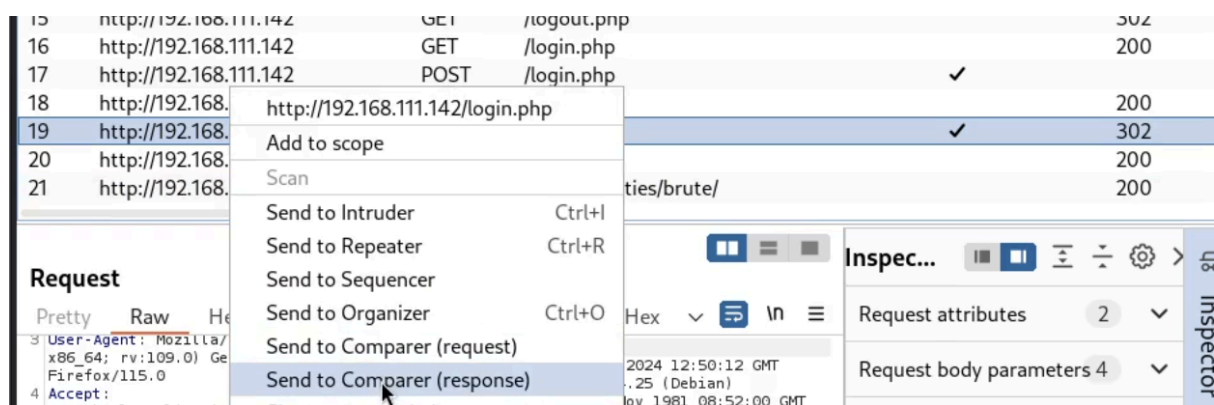
La siguiente herramienta que queremos ver va a ser la herramienta del comparer.

Esta herramienta sirve para comparar dos respuestas similares o dos peticiones que aparentemente son similares y que no vemos a la primera la diferencia.

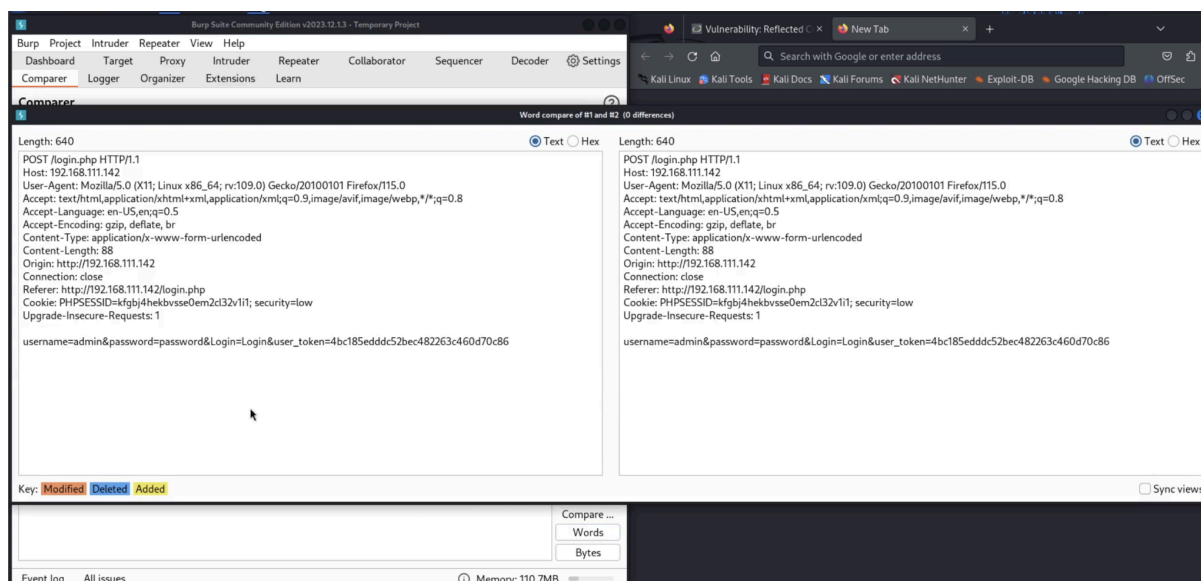
Fijaros, aquí arriba tenemos un ejemplo de dos content-length muy parecidos, de 85, 83 y queremos ver la diferencia.

En este caso es únicamente en el nombre y la contraseña y también en el token que es diferente y aquí nos lo marca con diferentes colores para que lo veamos.

Entonces a nivel de la herramienta nos iríamos al comparar y aquí podríamos cargarlo, pero lo más fácil es que desde el proxy o bien desde el history dijéramos por ejemplo este login de antes, este es el de admin, password, oye, mándame esto al comparar y mándame la petición y mándame esto al comparar y mándame otra vez la petición. Con botón derecho y send to comparer, poniendo si queremos la petición o la respuesta.



Ahora mismo tenemos dos peticiones iguales que ocupan lo mismo y digo vale, es que no sé si son iguales, compáralo a nivel de words y me dice que es lo mismo.



Entonces podríamos ver si una petición es igual o no, o si la respuesta es similar o no si cambiamos los datos que le enviamos nos lo permite ver en base a esos colores para diferenciarlo, si hay algo borrado, algo añadido o algo modificado.

En este caso la petición es exactamente la misma y no nos sacaría ningún color marcado.

Funcionalidad: Intruder

La parte más interesante junto al proxy dentro de las funcionalidades que tiene Burp sería el Intruder.



El Intruder es una herramienta dentro de Bur que nos va a permitir automatizar ataques más personalizados contra esas aplicaciones web y nos va a permitir identificar y explotar vulnerabilidades.

¿A qué vulnerabilidades me refiero?

Con esta herramienta vamos a poder hacer fuerzas brutas, vamos a poder hacer también fuerza bruta a la hora de identificar SQL Injection, a la hora de encontrar un cross site, hacer un buffer overflow, enumerar identificadores, etc.

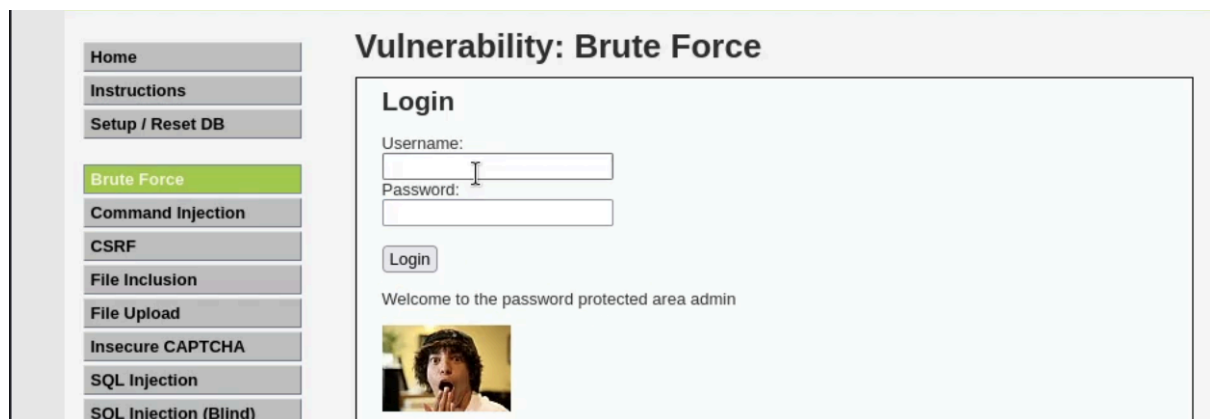
Esta es la funcionalidad más potente precisamente porque nos va a permitir hacer esto.

¿Qué pasa? Que la versión gratuita lo que tiene es que está limitada en cuanto al tiempo, pero vamos a verlo mejor con un ejemplo, porque de momento no se ha dicho exactamente lo que hace.

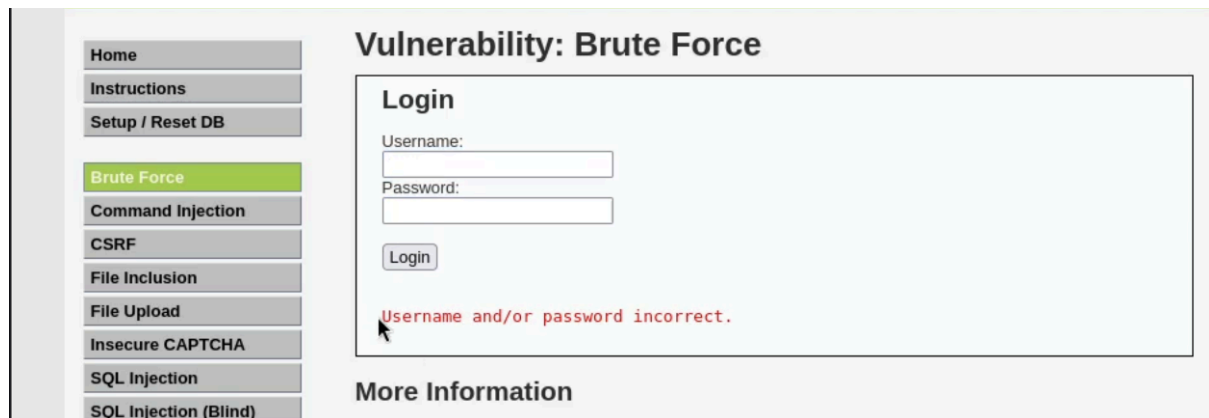
¿Cómo automatizamos estos ataques?

Nos vamos a ir de nuevo a la máquina virtual y nos vamos a ir a esta parte de la web DVWA que nos dice fuerza bruta.

Imaginaros para que veáis cómo funciona, si yo aquí pongo admin y password me dice que bienvenido al área protegida.

The image shows a web application interface for a security tool. On the left is a vertical sidebar with a list of menu items: Home, Instructions, Setup / Reset DB, Brute Force (highlighted in green), Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, and SQL Injection (Blind). The main content area has a title 'Vulnerability: Brute Force'. Below the title is a 'Login' section with two input fields labeled 'Username:' and 'Password:'. A 'Login' button is positioned below the password field. Under the button, a message reads 'Welcome to the password protected area admin' next to a small image of a person with a surprised expression.

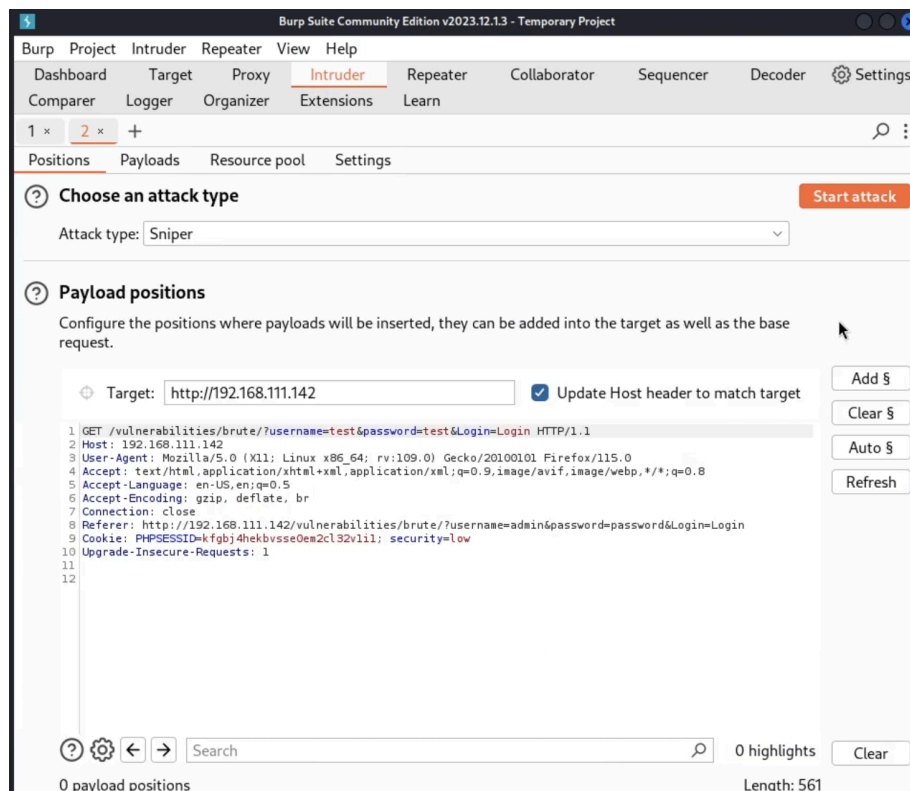
Vale, yo ya sé cuál es el usuario y cuál es la contraseña, imaginaros que no lo sé, entonces pongo aquí test / test, me dice el usuario con la contraseña es incorrecto.



Esto es lo primero que tendremos que hacer en la aplicación de manera manual, ver cómo funciona, cuál es su funcionamiento aparentemente esperado.

Hemos capturado estas peticiones mientras estamos solamente navegando por la herramienta, el proxy debería de estar configurado en un segundo plano.

Si ahora me voy al histórico, me voy a buscar esta petición en la que le he dicho que es test y test y esta botón derecho me la mando al intruder.



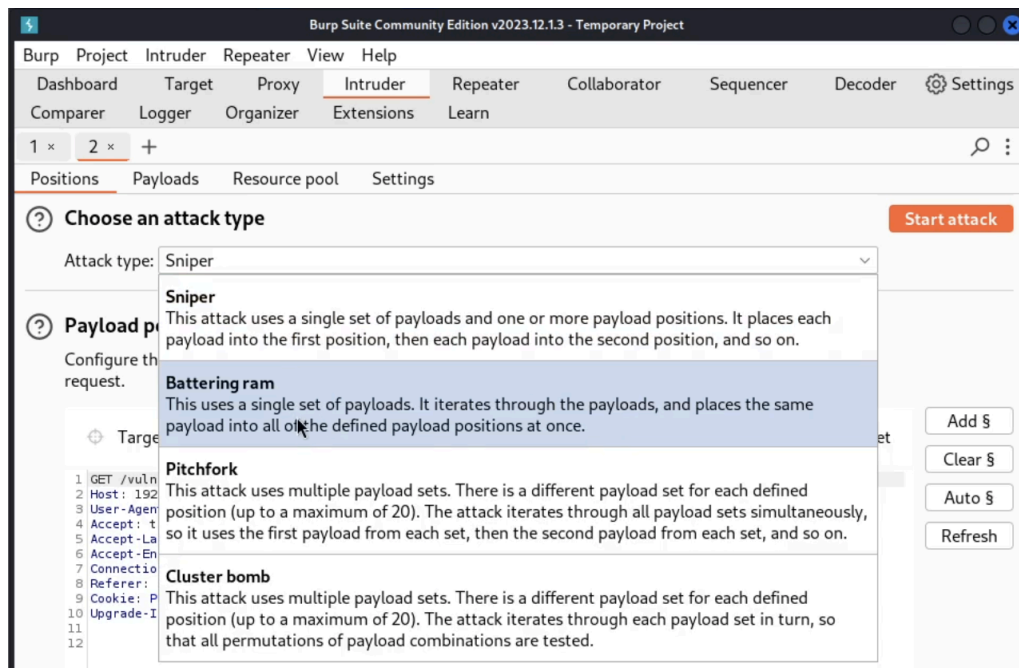
Este es el aspecto que tiene el intruder, aquí voy a decirle dónde quiero cargar payloads, es decir, me va a permitir hacer peticiones de manera automatizada, cambiando partes de la petición.

En este caso imaginamos que yo aquí sí que sé que el usuario es admin, pero no sé cuál es la contraseña, entonces quiero que las peticiones o ese payload se ejecute sobre la contraseña y poder probar muchas contraseñas lanzando muchas peticiones, pero yo solamente le doy un diccionario, entonces yo aquí lo he marcado con este simbolito que se puede añadir o borrar. Le he dado al Add seleccionando antes test para que se marque el password.



Además tiene diferentes tipos de ataques: sniper, battering ram, pitchfork o cluster bomb.

El más usual es el sniper, en el que todos los parámetros que se seleccionan se van a probar todas las opciones posibles del payload y una vez finalizado toma el siguiente valor y sigue el siguiente valor.



Luego tendríamos el battering ram, nos lo pone aquí en inglés, que vamos a cargar un único payload independientemente de los parámetros que haya, es decir, si yo lo pongo en admin y en password va a probar los mismos parámetros, la misma palabra en cada uno de los campos, es decir, va a probar admin / admin, test / test, hola / hola y no nos va a dar esa flexibilidad. Sin embargo en este caso el sniper lo configuro en password y va a hacer tantas peticiones como líneas tenga nuestro diccionario.

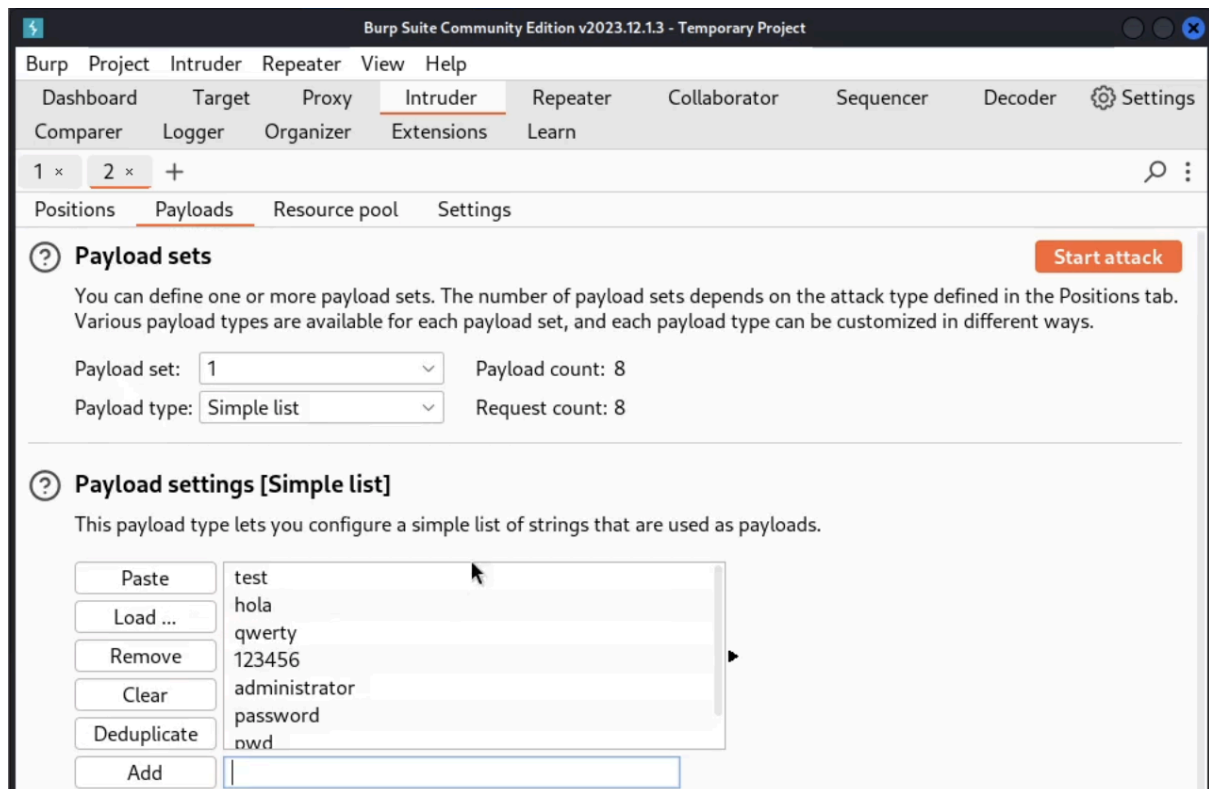
El pitchfork se prueba cada entrada del payload con su orden, este yo no lo he usado nunca a nivel de trabajo, imagínate que tengo dos campos, ya doy dos diccionarios, el primer valor del primer diccionario y el primero del segundo, el primero en buena variable, el segundo con el tercero, el tercero con el tercero, el cuarto con el cuarto, el quinto con el quinto y eso no me permite hacer una fuerza bruta total.

De hecho para hacer esa fuerza bruta total de probar todos con todos sería el cluster bomb, dónde tenemos varias posiciones y tenemos varios diccionarios, se prueban todos los valores de un diccionario con todos los valores del otro.

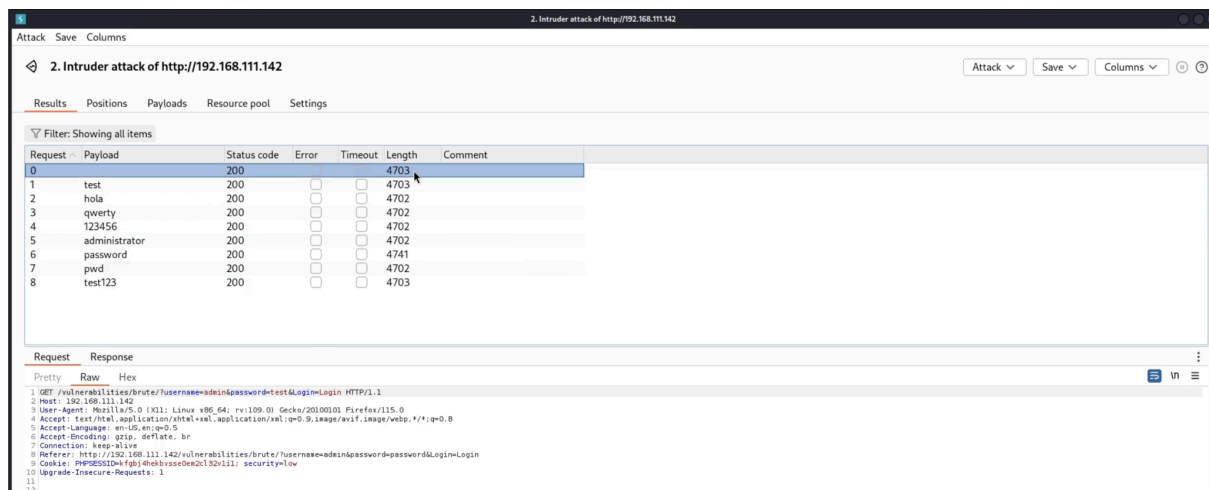
Así que hacer un cluster bomb sería la manera de hacer la fuerza bruta más bestia que haga todas las posibles peticiones.

El sniper es el más común cuando vamos a utilizar o vamos a atacar únicamente una posición del payload, o el user o el payload porque por ejemplo ya conocemos uno u otro, que es lo que vamos a hacer ahora.

Nos iríamos a la tab de payload y aquí tendríamos que cargarlo en la versión gratuita tenemos aquí como listas, aquí teníamos o bien que cargarlo de un archivo de un txt, que sería un diccionario que tengamos, o bien meterlo a mano. En este caso voy a meterlo a mano para que veáis cómo funciona, metemos test, metemos hola, metemos QWERTY, metemos 1 2 3 4 5 6, metemos Administrator, metemos password, metemos pwd, metemos test y hemos metido 8 contraseñas.



Va a probar estas 8 contraseñas en la misma petición, le daremos a Start Attack, me dice que como es el community va a ser más lento, que hay una limitación de tiempo, le digo que OK y fijaros, esto ha terminado.



Aquí yo ya veo que la respuesta esperada en este caso es la de "Username and/or password incorrect", y tiene una longitud de 4.703, vamos a ver 4.702 que también hay un match y de repente uno tiene más longitud, el password "password" que tiene 4741 que es justo el que en este caso funciona.

Yo busco esto por palabra incorrect y ya no me aparece, y si lo buscamos a mano a ver qué es lo que vemos raro, de repente me dice “Welcome to the password protected area”. Yo aquí ya puedo ver por la longitud que hay algo diferente y lo analizaría.

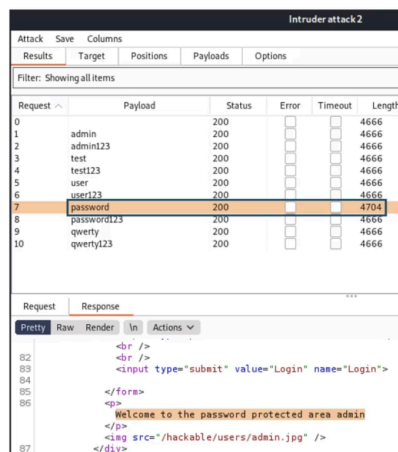
También lo podríamos ver por el código de error o por alguna cosa que sea diferente al resto de peticiones que hemos probado a mano, por eso a mí me gusta meter algunas a mano que yo he probado y luego meterle diccionarios.

Esta función es la más completa, ya os digo, fuerzas brutas, aquí podríamos meter payloads, de SQL injection, de cross site, descriptive, lo que estemos buscando en cada momento y es la más potente.

Intruder

• Steps:

- Send request to the intruder
- Select the variable
- Configure the payload
- Start attack
- Analyze responses



Request	Payload	Status	Error	Timeout	Length
0		200			4666
1	admin	200			4666
2	admin123	200			4666
3	test	200			4666
4	test123	200			4666
5	user	200			4666
6	user123	200			4666
7	password	200			4704
8	password123	200			4666
9	qwerty	200			4666
10	qwerty123	200			4666

Request	Response
82	
83	
84	<input type="submit" value="Login" name="Login">
85	</form>
86	<p>
87	Welcome to the password protected area admin
	</p>
	</div>

Los pasos os los he dejado también aquí identificados: seleccionamos la petición que nos interese, la mandamos al Intruder, seleccionamos la variable, configuramos el payload, comenzamos el ataque y analizamos las respuestas.

Sería la forma de proceder.

Como conclusiones, espero que ya hayáis empezado a ver la potencia que tiene esta herramienta de burbuja a la hora de encontrar vulnerabilidades, de explotaras y de trabajar dentro de las auditorías web.

Y además el hecho de que hay diferentes herramientas en una hemos visto el Repeater, hemos visto el Comparer, hemos visto el Intruder, hemos visto un poquito todos el Decoder y hemos visto sobre todo qué potencia tiene la herramienta a la hora de realizar las auditorías de seguridad web, que les va a permitir encontrar

vulnerabilidades en menos tiempo, de una manera más organizada, pudiendo ver en detalle cómo funciona la aplicación.

Así que herramienta que necesitamos aprender a utilizar, súper recomendada.